

Resumen

En este Trabajo Fin de Grado se diseña y desarrolla una aplicación web que permite determinar, mediante análisis forense automático, si un vídeo ha podido ser generado o manipulado por IA. El sistema combina un modelo de aprendizaje automático entrenado con XGBoost y un sistema ponderado que evalúa múltiples métricas extraídas de cada vídeo, correspondientes a las siguientes categorías: compresión, frecuencia espectral, ruido de sensor, movimiento óptico, textura de gradientes y metadatos.

El modelo se entrena con dos *datasets*, uno de vídeos reales con diferentes orígenes (móvil, Instagram, X...) y otro de vídeos generados por inteligencia artificial. En la carpeta de vídeos reales, se realiza esta división debido a métricas que se pueden ver afectadas por la recodificación de plataformas como Instagram o X, buscando evitar falsos positivos.

La parte principal del análisis se basa en las propiedades físicas del ruido de sensor que no se pueden manipular, como la correlación entre canales RGB o la planitud espectral del residuo de ruido. Para compensar los artefactos propios de cada plataforma, el sistema incorpora la lectura de metadatos generados en un reporte al descargar los vídeos procedentes de URL con yt-dlp y así aplicar ponderaciones diferenciadas durante el entrenamiento, para que métricas ligadas a la compresión y frecuencia se tengan menos en cuenta en estos casos.

El resultado es una herramienta accesible para el público general en la cual podrán apoyarse ante la duda causada por el reciente crecimiento de los vídeos generados por inteligencia artificial que circulan cada vez por redes sociales.

Resum

En aquest Treball de Fi de Grau es dissenya y desenvolupa una aplicació web que permet determinar, mitjançant anàlisis forense automàtica, si un vídeo ha pogut ser generat o manipulat per IA. El sistema combina un model d'aprenentatge automàtic entrenat y comparat amb (poner los diferentes modelos XGBoost, Random Forest, Roc-Auc) i un sistema ponderat que avalua múltiples mètriques extretes de cada vídeo corresponent a les categories següents: compressió, freqüència espectral, soroll de sensor, moviment òptic, textura de gradients y metadades.

El model s'entrena amb dos *datasets*, un de vídeos reals amb diferents orígens (móvil, Instagram, X...) i un altre de vídeos generats per intel·ligència artificial. Aquesta divisió ens dona les mètriques necessaries separades per origen per a preparar un model de predicció global y tres específics degut a la recodificació agresiva causada per la publicació a xarxes socials, tractant de reduir la quantitat de falsos positius.

La part principal de l'anàlisi es basa en les propietats físiques del soroll de sensor que no es poden manipular, com la correlació entre canals RGB o la planitud espectral del residu de soroll. Per a compensar els artefactes propis de cada plataforma, el sistema incorpora la lectura de metadades generades en un informe en descarregar els vídeos procedents d'URL amb yt-dlp i així aplicar ponderacions diferenciades durant l'entrenament, perquè les mètriques lligades a la compressió i la freqüència es tinguin menys en compte en aquests casos.



El resultat és una eina accessible per al públic general en la qual podran recolzar-se davant del dubte causat pel recent creixement dels vídeos generats per intel·ligència artificial que circulen cada vegada més per les xarxes socials.

Abstract

This project presents the design and development of a web application that determines, through automated forensic analysis, whether a video may have been generated or manipulated by AI. The system combines a machine learning model trained with XGBoost and a weighted scoring system that evaluates multiple metrics extracted from each video, corresponding to the following categories: compression, spectral frequency, sensor noise, optical flow, gradient texture, and metadata.

The model is trained on two *datasets*: one consisting of real videos from various sources (mobile, Instagram, X...) and another of AI-generated videos. Within the real video folder, this division is made due to metrics that can be affected by the re-encoding performed by platforms such as Instagram or X, in order to avoid false positives.

The core of the analysis relies on the physical properties of sensor noise that cannot be manipulated, such as the correlation between RGB channels or the spectral flatness of the noise residual. To compensate for the compression artifacts inherent to each platform, the system incorporates the reading of metadata generated in a report when downloading videos from URLs using yt-dlp, thereby applying differentiated weights during training so that metrics tied to compression and frequency are given less importance in these cases.

The result is a tool accessible to the general public, which they can turn to when in doubt about the growing number of AI-generated videos increasingly circulating on social media.



Agradecimientos

Le quiero dedicar este trabajo y esta montaña rusa que ha sido la universidad a mis padres y mi hermana, porque son el núcleo de mi vida y las personas que ríen conmigo, pero que también aguantan mis rebotes.

Papá, gracias por sacarme una sonrisa siempre que puedes y por disfrutar conmigo hasta de las mayores chorradas.

Ari, gracias por aguantar mi energía y ser la que pone la calma de los dos hermanos. Ojalá consigas todo lo que te estás proponiendo igual que lo estoy intentando yo.

Y en especial quería agradecer a mi madre.

Mamá, gracias por ser mi lugar seguro, el espejo en el que reflejarme y por acompañarme en los malos momentos, cuando había perdido toda confianza en mí mismo. Por no permitir que me hundiera y ser siempre el hombro en el que apoyarme. Todo lo que hago es para intentar que estés orgullosa de mí.

Como voy a olvidarme de mis compañeros y amigos. Gracias por haberme dejado compartir una etapa tan bonita, frustrante y divertida con vosotros, no la olvidaré nunca.

Y, por último, un agradecimiento especial a José Manuel, mi tutor del Trabajo de Fin de Grado. Desde la primera clase que nos diste sentí que eras una buena persona, y además de eso, un gran docente. Gracias por tener siempre una sonrisa y transmitir esa confianza que ayuda a que las cosas salgan bien y a sentirte seguro.

RESUMEN EJECUTIVO

La memoria del TFG del Grado en Tecnología Digital y Multimedia debe desarrollar en el texto los siguientes conceptos, debidamente justificados y discutidos, centrados en el ámbito de las tecnologías digitales y multimedia

CONCEPT (ABET)	CONCEPTO (traducción)	¿Cumple? (S/N)	¿Dónde? (páginas)
1. IDENTIFY:	1. IDENTIFICAR:		
1.1. Problem statement and opportunity	1.1. Planteamiento del problema y oportunidad	S	1-4
1.2. Constraints (standards, codes, needs, requirements & specifications)	1.2. Toma en consideración de los condicionantes (normas técnicas y regulación, necesidades, requisitos y especificaciones)	S	4-5
1.3. Setting of goals	1.3. Establecimiento de objetivos	S	5-6
2. FORMULATE:	2. FORMULAR:		
2.1. Creative solution generation (analysis)	2.1. Generación de soluciones creativas (análisis)	S	16 y 20-34
2.2. Evaluation of multiple solutions and decision-making (synthesis)	2.2. Evaluación de múltiples soluciones y toma de decisiones (síntesis)	S	13, 27-28 y 38-39
3. SOLVE:	3. RESOLVER:		
3.1. Fulfilment of goals	3.1. Evaluación del cumplimiento de objetivos	S	50
3.2. Overall impact and significance (contributions and practical recommendations)	3.2. Evaluación del impacto global y alcance (contribuciones y recomendaciones prácticas)	S	48 y 50-52



Índice

Capítulo 1.	Introducción	1
1.1	Contexto y justificación del trabajo	1
1.1.1	El auge de los vídeos sintéticos	1
1.1.2	Problema, desinformación, fraude y erosión de la confianza	2
1.1.3	Estado actual de las soluciones y oportunidad	3
1.2	Marco regulatorio	4
1.3	Objetivos del trabajo	5
1.3.1	Objetivo principal	5
1.3.2	Objetivos técnicos	5
1.3.3	Objetivos secundarios	6
1.4	Planificación del trabajo	6
1.5	Relación con las asignaturas del Grado GTDM	6
1.6	Relación con los ODS de la Agenda 2030 de la ONU	7
Capítulo 2.	Marco teórico	9
2.1	Vídeo sintético: modelos generativos	9
2.1.1	De las redes generativas adversariales a los modelos de difusión	9
2.1.2	Del modelo de imagen al modelo de vídeo	9
2.1.3	Modalidades de manipulación audiovisual	10
2.2	Análisis forense de vídeo	10
2.2.1	Metadatos y trazabilidad	10
2.2.2	Análisis espacial	11
2.2.3	Análisis temporal	11
2.2.4	Ruido de sensor y huella física	12
2.2.5	Aberración cromática y fotometría	12
2.2.6	Análisis facial	12
2.3	Machine learning para clasificación binaria	13
2.3.1	Comparación de algoritmos	13
2.3.2	Métricas de evaluación	13
2.3.3	Validación cruzada, sesgo de dominio y LOPO	14
2.4	Modelos multimodales	14
2.5	Explicabilidad: SHAP	15
Capítulo 3.	Metodología y material empleado	16
3.1	Metodología	16
3.1.1	Fases del proyecto	16
3.2	Material empleado	16



3.2.1	Lenguaje de programación y librerías	17
3.2.2	Procesado de vídeo e imagen	17
3.2.3	API externa.....	18
3.2.4	Aplicación web.....	18
3.2.5	Dataset	19
Capítulo 4.	Desarrollo del proyecto	20
4.1	Funcionamiento general.....	20
4.2	Construcción del <i>dataset</i>	21
4.2.1	Estructura y etiquetado	21
4.2.2	Fuentes.....	21
4.2.3	Extracción de features para entrenamiento.....	21
4.3	Extracción de <i>features</i> forenses	22
4.3.1	Análisis de nitidez	22
4.3.2	Análisis en el dominio de la frecuencia (FFT)	22
4.3.3	Flujo óptico.....	23
4.3.4	Análisis fotométrico	23
4.3.5	Aberración cromática	24
4.3.6	Análisis DCT y patrones de compresión	24
4.3.7	Análisis estadístico del ruido.....	25
4.3.8	Correlación intercanal RGB	25
4.3.9	Estructura IPB	26
4.3.10	Metadatos	27
4.3.11	Análisis facial.....	27
4.4	Entrenamiento del modelo unificado	27
4.4.1	Comparación de clasificadores.....	27
4.4.2	Evaluación: CV estratificada y LOPO.....	28
4.4.3	Persistencia del modelo	28
4.5	Modelos especializados por plataforma.....	28
4.6	Enrutamiento de modelos	28
4.7	Pipeline forense integrado	28
4.8	Análisis de contexto con Gemini	29
4.9	Explicación con SHAP	30
4.10	Aplicación web	30
4.10.1	Backend.....	31
4.10.2	Frontend	31
Capítulo 5.	Resultados y validación.....	35
5.1	Composición del <i>holdout</i>	35



5.2	Evolución del protocolo de evaluación.....	35
5.3	Precisión – AUC	38
5.4	Diagnóstico de sesgo de dominio (LOPO) y resultados del modelo unificado	39
5.5	Precisión – Recall	41
5.6	Matrices de confusión.....	42
5.7	Importancia de métricas.....	44
5.8	Discusión de resultados	48
Capítulo 6.	Conclusiones y trabajo futuro	49
6.1	Reflexión personal.....	49
6.2	Cumplimiento de objetivos.....	50
6.3	Aportaciones e impacto	50
6.4	Limitaciones del sistema.....	51
6.5	Trabajo futuro	51
6.5.1	Extensión de navegador.....	51
6.5.2	Robustez contra compresiones agresivas y ataques	51
6.5.3	Análisis facial	51
6.5.4	Incorporación de análisis de audio	52
6.5.5	Fusión del veredicto	52
Capítulo 7.	Bibliografía	53
Capítulo 8.	Anexos	56
8.1	Construcción del <i>dataset</i> – <i>extraer_datos.py</i>	56
8.1.1	Detección de la plataforma de origen.....	56
8.1.2	Extracción de fotogramas y lectura en memoria	56
8.1.3	Procesamiento de los vídeos.....	57
8.2	Entrenamiento del modelo general – <i>entrenar_ia.py</i>	57
8.2.1	LimpiadorFeatures.....	58
8.2.2	Eliminación del origen como dato para el entrenamiento	58
8.2.3	Clasificadores	59
8.2.4	Validación cruzada estratificada.....	59
8.2.5	LOPO.....	60
8.3	Modelos por plataforma – <i>entrenar_especializados.py</i>	60
8.3.1	Filtrar por origen.....	60
8.3.2	Índice de modelos.....	61
8.4	Enrutar modelos.....	61
8.5	Flujo principal y explicabilidad – <i>main.py</i>	61
8.5.1	Detección de origen en producción	62
8.5.2	Carga del modelo y predicción.....	62



8.5.3	Explicación SHAP.....	63
8.6	Aplicación web	63
8.6.1	Activación de eventos.....	63
8.6.2	Análisis de contexto con Gemini y prompt utilizado	65
8.6.3	Descarga desde URL con yt-dlp.....	66
8.7	<i>Endpoints</i> importantes – app.py.....	66
8.7.1	Análisis en segundo plano	67
8.7.2	Canal de Server Sent Events.....	68
8.8	Módulo de análisis facial (Fase Beta).....	68
8.8.1	Eye Aspect Ratio y conteo de parpadeos – análisis_facial.py.....	68
8.8.2	Calcular features faciales.....	69
8.8.3	Cálculo de features	70

Índice de figuras

Figura 1. <i>Frame</i> de vídeo real	1
Figura 2. <i>Frame</i> de vídeo sintético.....	2
Figura 3. Declaración <i>fake</i> Zelensky	2
Figura 4. ODS de la ONU	8
Figura 5. Artefacto de tablero de ajedrez	9
Figura 6. Interfaz página principal	21
Figura 7. <i>Features</i> medidas de un vídeo	22
Figura 8. Cálculo de parámetros de nitidez	22
Figura 9. Cálculo FFT	23
Figura 10. Cálculo del flujo óptico.....	23
Figura 11. Análisis fotométrico.....	24
Figura 12. Cálculos de aberración cromática	24
Figura 13. Cálculos bloques DCT	25
Figura 14. Análisis del ruido residual	25
Figura 15. Correlaciones canales de color.....	26
Figura 16. Cálculos estructura IPB	26
Figura 17. Lectura de metadatos	27
Figura 18. Veredicto análisis de contexto en vídeo generado por IA	30
Figura 19. Barra de progreso.....	31
Figura 20. Sección pegar URL para análisis y selección de fragmento de vídeo	31
Figura 21. Veredicto completo de vídeo real	32
Figura 22. Veredicto completo de vídeo <i>fake</i>	33
Figura 23. Interfaz de la aplicación en móvil.....	34
Figura 24. Rendimiento por plataforma (umbral 0.5)	36
Figura 25. Métricas globales (umbral 0.5)	36
Figura 26. Distribución por plataforma (umbral 0.5).....	37
Figura 27. Matriz de confusión global (umbral 0.5)	37
Figura 28. AUC global (umbral 0.4)	38
Figura 29. Rendimiento por plataforma (umbral 0.4)	39
Figura 30. Resultados evaluación LOPO	40
Figura 31. Rendimiento por plataforma modelo unificado (umbral 0.5)	40
Figura 32. Matriz de confusión modelo unificado	41
Figura 33. Recall global (umbral 0.4)	41
Figura 34. Matriz de confusión global (umbral 0.4)	43

Figura 35. Distribución de probabilidades de IA por clase (umbral 0.4)	43
Figura 36. Matrices de confusión por plataforma (umbral 0.4)	44
Figura 37. Métricas globales (umbral 0.4)	44
Figura 38. Importancia <i>features</i> modelo unificado	45
Figura 39. Importancia <i>features</i> Instagram	46
Figura 40. Importancia <i>features</i> Grabaciones/Descargas	47
Figura 41. Importancia <i>features</i> Twitter	47
Figura 42. Leer origen del vídeo	56
Figura 43. Extracción y lectura de <i>frames</i>	56
Figura 44. Función para procesar vídeos	57
Figura 45. Eliminar origen como parámetro de entrenamiento	58
Figura 46. Clasificadores binarios	59
Figura 47. Evaluación validación cruzada estratificada	59
Figura 48. Evaluación LOPO	60
Figura 49. Función para filtrar vídeos por origen	60
Figura 50. Creación índice de modelos	61
Figura 51. Función para llamar modelos en base al origen	61
Figura 52. Función para detección de origen en análisis	62
Figura 53. Funciones de carga y predicción con modelo	62
Figura 54. Función para la explicación de <i>features</i> con SHAP	63
Figura 55. Activación de eventos de análisis 1	64
Figura 56. Activación de eventos de análisis 2	65
Figura 57. <i>Prompt</i> y análisis de contexto con Gemini	65
Figura 58. Función de descarga de vídeo a analizar a partir de URL	66
Figura 59. Hilo de ejecución de análisis en segundo plano	67
Figura 60. Transmisión de eventos de análisis	68
Figura 61. Funciones base de análisis facial	68
Figura 62. Extracción de <i>features</i> faciales	69
Figura 63. Cálculo de <i>features</i>	70

Índice de siglas

- **API:** Application Programming Interface
- **AUC:** Area Under the Curve
- **CNN:** Convolutional Neural Network
- **CSS:** Cascading Style Sheets
- **CSV:** Comma-Separated Values
- **CV:** Cross-Validation
- **DCT:** Discrete Cosine Transform
- **DDPM:** Denoising Diffusion Probabilistic Models
- **DiT:** Diffusion Transformer
- **EAR:** Eye Aspect Ratio
- **FFT:** Fast Fourier Transform
- **Fps:** *frames* per second
- **GAN:** Generative Adversarial Network
- **GBM:** Gradient Boosting Machine
- **GOP:** Group of Pictures
- **HTML:** HyperText Markup Language
- **HTTP:** HyperText Transfer Protocol
- **IA:** Inteligencia Artificial
- **I/P/B (*frames*):** Intra/Predicted/Bidirectional *Frames*
- **JS:** JavaScript
- **JSON:** JavaScript Object Notation
- **LLM:** Large Language Model
- **LOOCV:** Leave-One-Out Cross-Validation
- **LOPO:** Leave-One-Platform-Out
- **LR:** Logistic Regression
- **ML:** Machine Learning
- **ODS:** Objetivos de Desarrollo Sostenible
- **PRNU:** Photo-Response Non-Uniformity
- **REST:** Representational State Transfer
- **RF:** Random Forest
- **RGB:** Red, Green, Blue
- **ROC:** Receiver Operating Characteristic
- **SDK:** Software Development Kit
- **SHAP:** Shapley Additive exPlanations
- **SSE:** Server-Sent Events
- **URL:** Uniform Resource Locator
- **VLM:** Vision-Language Model
- **XGBoost:** Extreme Gradient Boosting

Tablas

Tabla 1. Explicación de etiquetas.....	45
--	----

Capítulo 1. Introducción

1.1 Contexto y justificación del trabajo

1.1.1 El auge de los vídeos sintéticos

En los últimos cuatro años, la inteligencia artificial generativa ha pasado de ser un campo de investigación con resultados pobres y puramente académicos a una tecnología accesible al público general. Mientras que en 2020 las primeras redes generativas adversariales (GAN) producían imágenes faciales de calidad justa y fácilmente identificables como sintéticas, en 2024 la aparición de modelos de difusión aplicados al vídeo (teniendo Sora de OpenAI, que han cerrado recientemente, Veo de Google como referentes de las grandes empresas) han situado la generación de vídeo sintética en un punto de inflexión. Estos sistemas son capaces de generar secuencias de minutos de duración con resolución ya superior a 1080p, coherencia temporal entre *frames*, iluminación físicamente realista y movimiento de cámara humano. Esto lo completan los generadores comerciales (Kling, Runway, Luma, Pika, Seedance), que han convertido la generación de vídeo en un servicio accesible a través de suscripción o API. Un indicador del grado de integración audiovisual alcanzado es que, en febrero de 2026, cuatro de los principales modelos generaban audio correctamente sincronizado, cuando ninguno lo hacía a principios de 2025.

Esto ha llevado a que la barrera de entrada se haya reducido mucho. Generar un vídeo que hace cinco años habría exigido profesionales de postproducción con grandes equipos informáticos y horas de trabajo ahora es posible con un buen *prompt* en cualquier web. El coste de los vídeos de mayor calidad irá acorde a la suscripción pagada según la plataforma, un precio el cual puede resultarte barato teniendo en cuenta la similitud de calidad a la de un contenido real cuyo trabajo manual habría resultado más caro.



Figura 1. Frame de vídeo real



Figura 2. Frame de vídeo sintético

1.1.2 Problema, desinformación, fraude y erosión de la confianza

La consecuencia de esta expansión es la aparición de un volumen creciente de contenido sintético. Las redes sociales (Instagram, TikTok, X) se han convertido en el canal principal de distribución, y su proceso de recodificación agresiva camufla buena parte del rastro técnico que originalmente permite identificar un vídeo en crudo generado por IA. El problema no es ya, por tanto, únicamente la capacidad de generar, sino la capacidad de un usuario promedio de discernir lo auténtico de lo sintético.

Los riesgos asociados a esta tecnología son múltiples y bien documentados.

En al ámbito político, se han registrado casos de vídeos sintéticos utilizados para suplantar a líderes de Estado, como el *deepfake* del presidente de Ucrania, Zelensky, pidiendo la rendición de sus tropas en 2022 [1] o el caso más reciente de la polémica sobre la aparición del gobernador de Israel Netanyahu en una cafetería tras un tiempo sin dejarse ver y el abierto debate de cuan cierto podía ser este vídeo.



Figura 3. Declaración *fake* Zelensky

En el ámbito financiero, el conocido como *CEO fraud*, en el cual se suplanta la identidad de directivos importantes en vídeos y conferencias, ya ha causado pérdidas millonarias, teniendo de ejemplo el caso de Arup Hong Kong en 2024, que resultó en pérdidas por valor de 25 millones de dólares [2]. En este caso un empleado de finanzas recibió un correo de unos delincuentes haciéndose pasar por uno de sus directores. Organizaron una videoconferencia donde usaron *deepfakes* de vídeo y voz para hacer creer al empleado que estaba con sus superiores, solicitándole en la misma una serie de transferencias confidenciales a diferentes cuentas.

En el ámbito personal, la generación no consentida de imágenes íntimas sintéticas (siendo el grupo más afectado mujeres y menores de edad) es ya un fenómeno extendido, especialmente desde la publicación abierta de Grok por Elon Musk. Por ejemplo, tenemos un caso muy sonado en España en el que el famoso cantante urbano JC Reyes publicó unos desnudos de la artista Rosalía en Instagram generados sintéticamente [3]. El problema principal se debe a que los marcos legales, en este caso, avanzan más despacio que el desarrollo de la tecnología. Más allá de los daños puntuales o permanentes a la imagen y salud mental de las personas, la consecuencia más grave a largo plazo está en la base: la fiabilidad y confianza en vídeos e imágenes como evidencia objetiva en los juicios se está erosionando.

La gravedad de la situación se ve empeorada por una realidad ya estudiada: las investigaciones recientes afirman que las personas ya no son capaces de determinar de forma fiable si una cara es real o generado por IA [4], lo que lleva al desarrollo de sistemas de detección.

Este contexto sitúa la detección de vídeo sintético como una necesidad técnica y social. Lejos de ser un problema cerrado, es un problema en auge y dónde una sola herramienta única no sería suficiente, puesto que es una carrera entre generadores y detectores en la que cada nuevo modelo generativo invalida parte del trabajo realizado por el anterior detector de manera cíclica.

1.1.3 Estado actual de las soluciones y oportunidad

Existen actualmente soluciones comerciales para la detección de contenido sintético: **Hive Moderation**, **Reality Defender**, **Sensity AI**, **Truepic** o **Deepware**, cada una con un enfoque y una especialización diferentes.

Hive Moderation ofrece sus servicios en la nube a través de una API REST que analiza tanto imagen y vídeo como audio y texto para devolver puntuaciones de confianza. Su sistema se basa en la combinación de dos modelos, uno para detectar imágenes provenientes íntegramente de generadores específicos como DALL·E o Midjourney, y otro para *deepfakes* donde busca rostros y determina si son reales o manipulados [5].

Reality Defender trabaja en el ámbito empresarial, dónde combina diferentes modelos simultáneamente para cubrir todo el contenido posible, como por ejemplo detección en tiempo real en centros de llamadas y reuniones por videoconferencia [6].

Sensity AI se especializa en una detección forense a nivel facial generando informes sobre vídeo, imagen y audio, analizando los movimientos faciales y el audio para detectar inconsistencias o anomalías. También trabaja con análisis y monitorización de amenazas relacionadas con identidad digital en general [7].

Por último, **Truepic** no es un detector como tal, más bien una herramienta de análisis de procedencia, centrándose en los metadatos para verificar la autenticidad de un contenido [8].

La mayoría de estos sistemas se apoyan en redes neuronales convolucionales (CNN), en las que operan rostro tras rostro para capturar hasta los artefactos faciales más finos. Como todo modelo de entrenamiento, dependerá mucho de los datos sobre los que entrene, lo que lleva a varias empresas a trabajar con equipos específicos para generar *deepfakes* de alta calidad en un entorno controlado para entrenar y evaluar los detectores. A pesar de todo esto, el paso a la aplicación en contenido real sigue siendo el principal problema u obstáculo.

Estas son las soluciones y propuestas más serias y maduras que tiene el mercado, pero presentan limitaciones que justifican el desarrollo de una propuesta alternativa y de enfoque diferente:

- **Son sistemas cerrados.** El usuario recibe una puntuación o un veredicto, pero no tiene acceso a las señales o evidencias que han llevado a esa decisión. Esta falta de

transparencia hace que sea poco escalable de cara a contextos forenses, periodísticos o judiciales, donde la justificación del resultado tiene que ser un requisito.

- **Son de acceso de pago** y, principalmente, dirigidas a clientes particulares y empresas.

A esto se suma una limitación de alcance: al trabajar sobre caras mediante CNNs, estos sistemas no tienen información para trabajar ante el contenido artificial que no tiene caras y dependen de la calidad de la detección facial previa. Por eso, este trabajo da un giro a ese planteamiento, pasando a trabajar con métricas que no dependan de la presencia de una cara o una persona, relegando el análisis facial a un módulo complementario que se activa solo cuando detecte un rostro en un primer plano en la mayoría de los *frames* del vídeo. De esta manera, pongo las huellas físicas como centro de la detección, ya que estas están presentes en todo el fotograma y en todos los fotogramas.

La oportunidad que aborda este Trabajo Fin de Grado gira en torno a estas situaciones. El sistema desarrollado combina un análisis forense de bajo nivel, fundamentado en propiedades físicas del vídeo: ruido de sensor, estructura de compresión, coherencia temporal, aberración cromática, con un clasificador de aprendizaje automático entrenado sobre un *dataset* propio y diferenciado por plataforma de origen, siendo entonces la base del entrenamiento *fakes* reales que circulan por redes sociales. Sobre esa base se incorpora una capa de explicación mediante SHAP y un análisis semántico de contexto mediante una API (Gemini), de modo que cada resultado está acompañado de su justificación. El veredicto se expone al usuario final a través de una aplicación web accesible, siempre con la intención de ser una guía de apoyo y no una verdad absoluta.

1.2 Marco regulatorio

El contexto normativo en el que se inscribe la detección de contenido sintético está experimentando una evolución acelerada durante los últimos años. Los principales condicionantes regulatorios son los siguientes:

En el plano europeo, el Reglamento (UE) 2024/1689, conocido como Ley de Inteligencia Artificial, que está en vigor desde el 1 de agosto de 2024, establece 50 obligaciones de transparencia que incluyen el marcado en formato legible por máquina del contenido generado por IA y la identificación clara de los *deepfakes*, aplicables a partir del 2 de agosto de 2026. El primer borrador del código fue publicado por la Comisión Europea en diciembre de 2025. Esta exigencia de trazabilidad del origen sintético como deber jurídico autónomo legitima el desarrollo de herramientas de verificación, que actúan en vídeos donde el etiquetado de origen no existe o ha sido eliminado.

A esta regulación se ha sumado una prohibición específica propuesta e impulsada por España, y acordada en mayo de 2026, la normativa incorpora como prácticas prohibidas los sistemas capaces de generar o manipular imágenes, vídeos o audio realistas de contenido sexual sin consentimiento, así como destinados a generar pornografía infantil.

En el plano nacional, el Consejo de Ministros aprobó en mayo de 2026 el proyecto de Ley para el buen uso y la gobernanza de la inteligencia artificial, que adapta la legislación española al reglamento europeo, atribuye a la Agencia Española de Supervisión de Inteligencia Artificial (AESIA) la vigilancia del cumplimiento y prevé sanciones de hasta 35 millones de euros. Con anterioridad, el Gobierno había aprobado en marzo de 2025 un proyecto de reforma del Código Penal para tipificar como delito los *deepfakes* de contenido sexual realizados sin consentimiento,

en el marco de la ley orgánica de protección de las personas menores de edad en los entornos digitales.

Finalmente, el funcionamiento del sistema desarrollado queda sujeto al Reglamento General de Protección de Datos (Reglamento (UE) 2016/679). Afecta de dos maneras: por una parte, el apartado de análisis facial procesa datos biométricos; por otra, la descarga y procesamiento de vídeos procedentes de redes sociales obligan a un tratamiento responsable del material, únicamente con la finalidad de verificación y sin conservación indebida.

1.3 Objetivos del trabajo

1.3.1 *Objetivo principal*

El objetivo principal del proyecto es diseñar, implementar y validar un sistema híbrido de detección de vídeo generado por inteligencia artificial, que combine análisis forense de bajo nivel con clasificación supervisada y razonamiento multimodal, y que ofrezca al usuario final un veredicto justificado y trazable a través de una interfaz web.

Este objetivo se aleja de la posibilidad de poder obtener un rendimiento lo más alto posible ya que no busco solo clasificar correctamente, sino hacerlo de manera que el porqué de cada cosa sea visible y que el rendimiento se mida con condiciones de uso real.

1.3.2 *Objetivos técnicos*

Para alcanzar el objetivo principal, el trabajo se ha desglosado en los siguientes objetivos técnicos concretos:

- **Construir un *dataset* propio y equilibrado** de vídeos reales y sintéticos, etiquetados por plataforma de origen (Instagram, X/Twitter, Tiktok, grabación directa desde móvil), con el fin de capturar el efecto real de la compresión y recodificación de datos sobre las trazas forenses.
- **Diseñar e implementar una capa de extracción de datos forenses** que incluya métricas espaciales (nitidez, FFT, DCT), temporales (flujo óptico, estructura IPB de la compresión), fotométricas (brillo, iluminación, aberración cromática), de ruido (asimetría, curtosis, gaussianidad mediante test de Shapiro-Wilk, correlación inter-canal RGB) y de metadatos (ExifTool, coherencia de marcas temporales, fabricante).
- **Entrenar y comparar varios modelos de clasificación supervisada** (Random Forest, Gradient Boosting, Logistic Regression, XGBoost) sobre los parámetros extraídos, evaluando con validación cruzada y diagnosticando el sesgo de dominio mediante una estrategia **Leave-One-Platform-Out (LOPO)**.
- **Desarrollar un esquema de modelos especializados por plataforma**, complementado con un enrutador inteligente y un modelo unificado como salvaguarda en caso de no conocer y no poder detectar el origen del vídeo, para mitigar el sesgo de dominio detectado.
- Añadir una segunda opinión inteligente: **un modelo visual (Gemini)** analiza la escena en su conjunto para comprobar que todo lo que aparece tiene sentido y encaja entre sí.

- No dar solo da un veredicto, sino explicar por qué llegó a esa conclusión, traduciendo los datos técnicos a un lenguaje claro que cualquier persona pueda entender, sin necesidad de saber de imagen y procesamiento de vídeo.
- **Construir una aplicación web** que presente el sistema completo y permita su uso directo al usuario, con soporte para análisis tanto de archivos locales como de vídeos provenientes de URL (descargados mediante yt-dlp).

1.3.3 *Objetivos secundarios*

Adicionalmente, el proyecto persigue una serie de objetivos secundarios con impacto más allá del propio desarrollo:

- Generar un sistema fácilmente escalable, de manera que la incorporación de nuevas plataformas, nuevas *features* o nuevos modelos no obligue a rehacer el conjunto.
- Documentar el flujo de trabajo de forma que el sistema pueda servir como base reproducible para futuros trabajos sobre detección forense de vídeo.
- Contribuir a la alfabetización mediática ofreciendo una herramienta accesible al público general no especializado.
- Incorporar un módulo de análisis facial complementario actualmente en fase beta que, cuando el video contiene un primer plano sostenido a lo largo del vídeo, evalúe señales fisiológicas y geométricas de la cara como el parpadeo, la simetría o temblores a modo de evidencia adicional al análisis forense.

1.4 Planificación del trabajo

El proyecto se ha desarrollado en cuatro grandes fases:

1. **Análisis del problema y construcción del *dataset*.** Definición del alcance, revisión bibliográfica del estado del arte, recopilación y etiquetado de vídeos reales y sintéticos de manera manual, organización del *dataset* por plataforma de origen, revisando manualmente los vídeos y descartando aquellos que puedan generar dudas de su origen.
2. **Diseño e implementación del pipeline forense.** Desarrollo de los módulos de extracción de *features*, integración con ExifTool y FFmpeg, junto con la validación de la coherencia de las métricas sobre muestras conocidas.
3. **Entrenamiento, evaluación y refinamiento de modelos.** Comparación de algoritmos de clasificación, análisis del sesgo de dominio mediante LOPO (explicar más adelante), desarrollo de modelos especializados por plataforma, integración del enrutador y de la capa multimodal con Gemini.
4. **Construcción de la aplicación web y documentación.** Implementación del servidor Flask, diseño del *frontend*, integración de SHAP, redacción de la memoria.

1.5 Relación con las asignaturas del Grado GTDM

Este Trabajo Fin de Grado integra de forma natural conocimientos adquiridos en una buena parte de las asignaturas del Grado en Tecnología Digital y Multimedia. Las que más directamente han sustentado el desarrollo son las siguientes:

- **Redes de distribución de contenidos:** aporta los fundamentos de codificación de vídeo, estructura de los *frames* I, P y B en los códecs intra/inter-cuadro, transformada DCT, análisis en el dominio de la frecuencia mediante FFT y aberración cromática, todos ellos elementos centrales del módulo forense del sistema.
- **Programación:** imprescindible para la arquitectura modular del pipeline, la gestión eficiente de la lectura única de *frames* en memoria y la organización del código en módulos reutilizables.
- **Imagen y Vídeo Digital:** fundamentan el diseño, entrenamiento y evaluación de los clasificadores supervisados (Random Forest, Gradient Boosting, XGBoost), las métricas de evaluación (ROC-AUC, F1, matriz de confusión) y los conceptos de sesgo de dominio y validación cruzada.
- **Tecnologías de Desarrollo Web:** necesarias para la construcción del *frontend* HTML/CSS/JavaScript y la comunicación con el servidor.

Más allá de estas asignaturas concretas, el trabajo recoge las competencias transversales del Grado: la capacidad de abordar un problema complejo combinando varias disciplinas, la gestión autónoma del proyecto, el uso de herramientas profesionales de desarrollo, y la comunicación técnica escrita.

1.6 Relación con los ODS de la Agenda 2030 de la ONU

El proyecto se alinea con varios de los Objetivos de Desarrollo Sostenible aprobados por las Naciones Unidas en 2015. A continuación, se describen las contribuciones más relevantes:

ODS 16 — Paz, justicia e instituciones sólidas Es la conexión más directa con este trabajo. Este objetivo busca, entre otras cosas, que la información pública sea fiable y que las libertades fundamentales estén protegidas. El vídeo sintético amenaza precisamente eso: cuando cualquiera puede fabricar un vídeo falso convincente, la conversación pública se envenena y la democracia se resiente. Tener una herramienta accesible para comprobar si un vídeo es auténtico ayuda a que la ciudadanía y las instituciones puedan defenderse mejor frente a la desinformación.

ODS 9 — Industria, innovación e infraestructura El proyecto aplica tecnologías digitales de primer nivel (visión por computador, aprendizaje automático, modelos multimodales) con el objetivo de hacer el entorno digital más seguro y verificable. Además, el enfoque metodológico adoptado (*dataset* propio, modelos adaptados al tipo de contenido, explicaciones basadas en SHAP) es perfectamente trasladable a otros problemas de clasificación en contextos industriales donde el origen de los datos importa.

ODS 4 — Educación de calidad De forma indirecta, una herramienta de verificación accesible es también una herramienta educativa. Cuando el sistema explica qué indicios le llevaron a su conclusión, el usuario aprende a mirar un vídeo de otra manera. Esto conecta directamente con lo que organismos como la UNESCO llevan años reclamando: la alfabetización mediática como competencia básica del siglo XXI.

ODS 5 — Igualdad de género Los estudios disponibles indican que más del 90% de los *fakes* que había en internet en 2020 eran pornografía, siendo la mayoría de las víctimas de generación no consentida de imágenes íntimas sintéticas mujeres y niñas [9]. Facilitar la detección y retirada de este tipo de contenido contribuiría a reducir una forma muy concreta de violencia de género en el entorno digital.

ODS 10 — Reducción de las desigualdades Hoy en día, las herramientas profesionales para verificar contenido están al alcance de medios de comunicación, grandes empresas e instituciones con presupuesto. El ciudadano de a pie no tiene acceso a nada parecido. Un sistema de código abierto y uso libre otorga una capacidad similar de verificación en manos de cualquier persona, independientemente de sus recursos.



Figura 4. ODS de la ONU

Capítulo 2. Marco teórico

2.1 Vídeo sintético: modelos generativos

2.1.1 De las redes generativas adversariales a los modelos de difusión

La historia moderna de la síntesis visual mediante inteligencia artificial comienza con la propuesta de las Generative Adversarial Networks (GAN) [10]. Una GAN consiste en dos redes neuronales entrenadas en oposición: un generador que produce muestras y un discriminador que intenta distinguirlas de muestras reales. El proceso adversarial fuerza al generador a producir salidas cada vez más realistas. Variantes posteriores como StyleGAN [11] y StyleGAN2 [12] llevaron la calidad de la síntesis facial a niveles mucho más realistas y similares a los de una cámara, y abrieron la primera oleada de *deepfakes* accesibles al gran público. A pesar del gran avance que representó como primer sistema moderno, este cuenta con una fuerte manera de delatarse, el famoso *checkerboard artifact* o patrón de ajedrez [13]. Cuando una GAN genera un video o una imagen empieza desde un código matemático aleatorio muy pequeño, el cual se va estirando hasta alcanzar resoluciones altas. Para ello, el generador superpone bloques de píxeles utilizando un tamaño de filtro y un paso de avance. Cuando el tamaño del filtro no es divisible por el paso de avance, el reescalado de la imagen dejará la siguiente huella espectral.

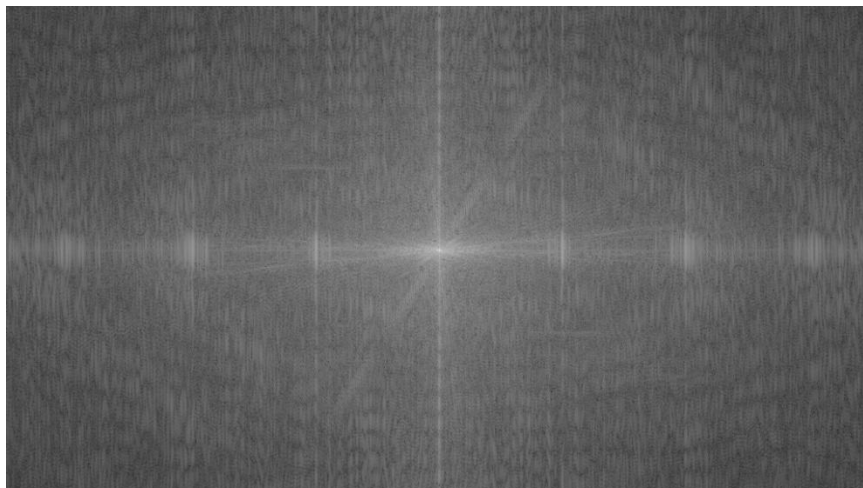


Figura 5. Artefacto de tablero de ajedrez

A partir de 2020, sin embargo, la dominancia en generación visual ha cambiado hacia los modelos de difusión (Diffusion Probabilistic Models) [14]. La idea es radicalmente distinta a la de las GAN: se aprende un proceso de "ruido inverso" capaz de reconstruir una imagen coherente a partir de ruido gaussiano puro, mediante un número finito de pasos de eliminación de ruido. La estabilidad del entrenamiento y la calidad de las muestras producidas convirtieron rápidamente a los modelos de difusión en el estado del arte. Stable Diffusion [15], al operar en el espacio comprimido por un *autoencoder* en lugar de en el espacio de píxeles, demostró que estos modelos podían ejecutarse con recursos modestos y democratizó su uso.

2.1.2 Del modelo de imagen al modelo de vídeo

Tanto los modelos GAN como los modelos de difusión empiezan con la generación de imágenes como base, siendo el dominio de vídeo un reto específico. La principal dificultad es la coherencia temporal: tan solo generar imágenes de calidad no asegura un vídeo de calidad, puesto que la secuencia debe respetar la continuidad del movimiento, la persistencia de los objetos o la consistencia de la iluminación. Los primeros intentos resolvían este problema mediante capas temporales adicionales sobre arquitecturas de difusión preentrenadas de imagen.

El verdadero punto de inflexión llegó en febrero de 2024 con la presentación de **Sora** por OpenAI [16], basada en una arquitectura de tipo *Diffusion Transformer* (DiT) [17] entrenada sobre "parches espaciotemporales" de vídeo. Sora demostró que era posible generar secuencias de hasta un minuto con resolución 1080p, coherencia temporal sostenida y comprensión semántica del *prompt* textual a un nivel que situaba la frontera entre lo real y lo sintético en un punto incierto frente a lo que ya conocíamos. A lo largo de 2024 y 2025, modelos como **Veo** y **Veo 3** de Google DeepMind, **Runway Gen-3 / Gen-4**, **Kling** de Kuaishou, **Hailuo** de MiniMax y **Sora 2** de OpenAI han consolidado esta nueva generación de sistemas, todos ellos basados en variantes de la arquitectura DiT.

2.1.3 Modalidades de manipulación audiovisual

Es importante hacer la distinción entre las diferentes modalidades de contenido sintético, dado que los detectores trabajaran de una manera u otra en base a lo que se busque:

- **Sustitución facial (face swap):** en un vídeo existente, se reemplaza la cara de una persona por la otra. Las trazas detectables se concentran en los bordes de la cara y las inconsistencias entre el rostro insertado y el cuerpo original.
- **Sincronización labial (lip sync):** se modifica únicamente el movimiento de la boca para que coincida con un audio dado.
- **Títere:** el rostro y los gestos de una persona se manipulan en tiempo real, controlados por otra persona. Se correspondería con el caso ya mencionado de Hong Kong.
- **Generación íntegra (text-to-video, image-to-video):** el vídeo se construye de 0 a partir de un *prompt* textual o de una imagen de base, sin necesidad de un vídeo original. Es el caso de modelos como Sora, Veo o Kling. Se puede combinar por ejemplo con la sincronización labial para generar un vídeo de ti mismo hablando a partir de una imagen de tu propia cara.

Este trabajo de fin de grado se enfoca principalmente en la última categoría, que es la más expandida y moderna, la que crece a mayor velocidad y en consecuencia la que peor cubren las herramientas de detección existentes.

2.2 Análisis forense de vídeo

El análisis forense multimedia es la disciplina que estudia cómo verificar la autenticidad y trazar el origen de un contenido audiovisual a partir de sus propiedades intrínsecas, sin recurrir a información externa. El principio sobre el que se sustenta toda la disciplina es que cada etapa del pipeline de captura, codificación y distribución deja una serie de huellas verificables en el archivo final. Una cámara real introduce trazas físicas (ruido de sensor, distorsión óptica, patrones de *demosaiing*); un códec real introduce trazas de compresión (estructura GOP, cuantización DCT); un editor introduce trazas de manipulación (recodificaciones múltiples, inconsistencias temporales). La ausencia o la incoherencia de estas trazas es, por sí misma, un indicio de manipulación.

2.2.1 Metadatos y trazabilidad

El primer nivel de análisis es el más superficial pero también el más eficiente: los metadatos del archivo. Los contenedores multimedia modernos (MP4, MOV, MKV) almacenan información sobre el dispositivo de captura, las fechas de creación y modificación, el software empleado, parámetros de codificación y, en el caso de imagen, datos EXIF detallados. La herramienta de referencia utilizada en este proyecto para la extracción de metadatos forenses es ExifTool, desarrollada por Phil Harvey, que soporta más de cien formatos de archivo y miles de etiquetas.

El valor forense de los metadatos reside tanto en su presencia como en su coherencia interna. Una grabación auténtica con un teléfono móvil presenta típicamente fabricante (Apple, Samsung), modelo concreto, fechas coherentes (creación anterior o igual a modificación) y software de cámara nativo. Un vídeo generado mediante IA, por el contrario, suele presentar metadatos ausentes, fabricantes genéricos (Adobe, Lavf, Synthesia), paradojas temporales como fechas de creación posteriores a las de modificación o incluso la tecnología de IA de la que provienen. Estas inconsistencias, sin ser pruebas definitivas, constituyen indicadores valiosos cuando se combinan con análisis de bajo nivel.

2.2.2 Análisis espacial

Cada *frame* de un vídeo puede analizarse como una imagen independiente, y de esta inspección surge un conjunto de métricas clásicas en visión por computador.

Nitidez basada en el operador laplaciano. Aplicar el operador laplaciano a un *frame* y calcular la varianza del resultado nos da una medida de la cantidad de bordes y, por tanto, de la nitidez global de la imagen. Una varianza baja indica una imagen poco enfocada, mientras que valores muy altos o diferentes entre *frames* consecutivos pueden indicar generación artificial: los modelos generativos suelen presentar una nitidez homogénea o, por el contrario, a presentar saltos abruptos entre *frames*.

Transformada de Fourier (FFT). La FFT permite analizar la distribución de energía de una imagen en el dominio de la frecuencia. Las imágenes naturales presentan típicamente una distribución de potencia que decae con la frecuencia siguiendo una ley aproximada $1/f^2$ [18]. Los vídeos generados por IA, concretamente los basados en arquitecturas convolucionales, tienden a dejar huellas detectables en este dominio: un exceso de energía en altas frecuencias asociado a artefactos de muestreo (*upsampling*) [19].

Transformada Coseno Discreta (DCT). La DCT es la base de la mayoría de los códecs modernos (JPEG, MPEG, H.264), que operan sobre bloques de 8×8 píxeles. Analizar la energía de la DCT a nivel de bloque permite detectar rastros de doble compresión y caracterizar el grado de degradación introducido por procesos de recodificación sucesivos.

2.2.3 Análisis temporal

A diferencia de una imagen aislada, el vídeo introduce una dimensión temporal de la que se pueden extraer una serie de evidencias.

Flujo óptico. El flujo óptico estima el desplazamiento aparente de cada píxel entre dos *frames* consecutivos, y constituye la herramienta fundamental para analizar el movimiento. El método de Farnebäck [20], implementado en OpenCV mediante `cv2.calcOpticalFlowFarneback`, es uno de los algoritmos densos más utilizados. Para detección forense, no interesa tanto la magnitud absoluta del flujo como su suavidad, sus discontinuidades y sus saltos anómalos: un movimiento excesivamente regular puede indicar interpolación artificial, mientras que saltos bruscos no justificados por un corte de plano apuntan a discontinuidades temporales generadas por el modelo.

Estructura IPB de la compresión. Los códecs modernos de vídeo utilizan tres tipos de *frames*: I-*frames* (codificados de forma independiente, *frames* clave de la secuencia), P-*frames* (codificados a partir de la diferencia con el *frame* anterior) y B-*frames* (predichos a partir de *frames* anteriores y posteriores). La estructura del Group of Pictures (GOP) es una propiedad característica del codificador y del proceso de captura. Un vídeo real grabado con un dispositivo móvil presenta una estructura GOP estable con presencia de B-*frames*; un vídeo exportado *frame* a *frame* por un generador de IA típicamente carece de B-*frames* o presenta un GOP anómalo. El análisis de la estructura IPB se realiza mediante FFprobe, parte del paquete FFmpeg. La estructura del GOP es un parámetro que se debe trabajar con cuidado, puesto que se puede ver afectado si el

vídeo ha sido publicado y descargado muchas veces en plataformas y redes sociales, de manera que puede presentar anomalías siendo un vídeo real.

2.2.4 Ruido de sensor y huella física

Sobre el papel, una de las contribuciones más relevantes del análisis forense de imagen es el concepto de *Sensor Pattern Noise* (SPN) o, más específicamente, *Photo-Response Non-Uniformity* (PRNU) [21]. Cada sensor CMOS o CCD presenta pequeñas imperfecciones de fabricación que generan un patrón de ruido único y persistente, lo podríamos comparar con la huella dactilar humana, pero en los sensores. Este ruido se preserva a través de las transformaciones habituales (compresión, redimensionado, ajustes de iluminación) y se utiliza tanto para vincular un vídeo a un dispositivo concreto como para detectar manipulaciones.

Para la detección de vídeo generado por IA, podemos aprovecharnos de estas huellas: no se busca identificar el sensor, sino comprobar que existe algún ruido con estructura compatible con un sensor real. Las características que se analizan son:

Gaussianidad del ruido residual. El ruido térmico y de lectura de un sensor real no es perfectamente gaussiano, pero presenta una distribución cercana con ciertas asimetrías. Un test de Shapiro-Wilk [22] aplicado al ruido residual de un vídeo generado por IA tiende a rechazar la hipótesis de no-gaussianidad con un p-valor anormalmente alto: el ruido del generador es demasiado perfecto.

Asimetría y curtosis. Los momentos estadísticos de tercer y cuarto orden de la distribución del ruido proporcionan información complementaria. El ruido real presenta colas más pesadas (curtosis positiva) y cierta asimetría, mientras que el ruido sintético tiende a la simetría y a la curtosis nula.

Correlación inter-canal. El proceso de *demosaiicing* (interpolación del filtro de Bayer para reconstruir los tres canales RGB a partir del único canal sensible del sensor) introduce correlaciones específicas entre canales adyacentes. Un vídeo generado en el espacio de color RGB sin pasar por este proceso presenta correlaciones inter-canal alteradas respecto a las propias de una cámara real.

2.2.5 Aberración cromática y fotometría

Las lentes reales presentan **aberración cromática lateral** [23], un fenómeno físico por el cual las componentes de distinto color (rojo, verde, azul) se enfocan en posiciones ligeramente diferentes del plano del sensor, generando un pequeño desplazamiento entre canales que crece radialmente desde el centro de la imagen. Este efecto es difícil de simular por un modelo generativo porque tendría que simular una óptica física real. Su ausencia o su distribución anómala son otro buen indicador.

Por último, el **análisis fotométrico** estudia la coherencia de la iluminación a lo largo del vídeo: dirección estimada de la fuente de luz, variación temporal del brillo, presencia de tendencias lineales (*fades* artificiales). Una variación de brillo no justificada por cambios reales en la escena, o una tendencia lineal sostenida a lo largo de todo el vídeo, son indicadores de procesamiento artificial.

2.2.6 Análisis facial

Cuando el vídeo tiene un rostro en primer plano, se activa el análisis complementario centrado en señales fisiológicas que un generador muchas veces reproduce incoherentemente a lo largo del tiempo. El estudio *In Ictu Oculi* [24] ha demostrado que el parpadeo representa un evento de discriminación eficaz; una frecuencia de parpadeo fuera del rango humano normal (15-20 parpadeos por minuto) puede ser un indicio de manipulación. Esto lleva a la idea de desarrollar

un módulo independiente en fase de prueba para que el usuario tenga libertad de elegir que análisis le convence más.

2.3 Machine learning para clasificación binaria

El conjunto de *features* forenses descritas constituye un vector de características que puede alimentar a un clasificador supervisado. La tarea es de clasificación binaria: si el vídeo es real el resultado será 0, si es sintético será 1.

2.3.1 Comparación de algoritmos

En este trabajo se han comparado cuatro familias de algoritmos, todos ellos disponibles en la librería scikit-learn [25]:

- **Regresión logística (LR)**. El modelo lineal de referencia. Ofrece interpretabilidad directa (coeficientes asociados a cada *feature*) pero está limitado a fronteras de decisión lineales. Sirve como *baseline*.
- **Random Forest (RF)** [26]. Construye un conjunto de árboles de decisión entrenados sobre subconjuntos aleatorios de muestras y *features* (bagging + *feature bagging*). El voto mayoritario reduce la varianza individual de cada árbol y proporciona robustez frente a *outliers* y a *features* ruidosas.
- **Gradient Boosting (GBM)** [27]. Técnica de construcción secuencial de modelos débiles, donde cada nuevo modelo se entrena para corregir los errores de los anteriores. A diferencia de Random Forest, la construcción es secuencial y aditiva.
- **XGBoost** [28]. Es una implementación altamente optimizada de gradient boosting que introduce regularización explícita, tratamiento eficiente de valores ausentes y paralelización.

La elección de algoritmos basados en árboles, en lugar de redes neuronales profundas, se debe a los siguientes aspectos: el tamaño del *dataset*, la heterogeneidad de las *features*, la mayor facilidad de interpretación mediante SHAP, la presencia de soluciones privadas ya desarrolladas a través de CNNs y la incapacidad del equipo utilizado en el desarrollo para el procesamiento de vídeo (más pesado que el de imagen) a través de redes neuronales.

2.3.2 Métricas de evaluación

No podemos reducir la evaluación de un sistema binario a una única cifra, por lo que se trabaja con diferentes métricas. Las métricas empleadas en este trabajo son:

Accuracy. Proporción de predicciones correctas sobre el total. Es fácil de interpretar, pero puede resultar engañosa en *datasets* desbalanceados.

Precision y Recall. La *precision* mide qué parte de los positivos predichos son realmente positivos; el *recall*, qué parte de los positivos reales han sido detectados. En detección de vídeo sintético hay un factor importante: priorizar precisión reduce falsos positivos (acusar a un vídeo real); priorizar *recall* reduce falsos negativos (dejar pasar un vídeo falso).

En este caso, busco el equilibrio entre las dos, pero dándole una pequeña prioridad a la precisión. Esto se realiza a través de la calibración manual del umbral.

F1-score. Media armónica de precisión y recall, útil como métrica única cuando se busca equilibrio entre ambas.

ROC-AUC. Área bajo la curva Receiver Operating Characteristic, que representa el balance entre tasa de verdaderos positivos y tasa de falsos positivos a lo largo de todos los umbrales de decisión posibles. Es la métrica más robusta para evaluar la calidad de la separación entre clases independientemente del umbral elegido.

Matriz de confusión. Representación del reparto completo de aciertos y errores por clase y totales.

2.3.3 Validación cruzada, sesgo de dominio y LOPO

La validación cruzada estratificada (Stratified K-Fold) es el procedimiento estándar para estimar el rendimiento de un modelo sobre datos no vistos: el *dataset* se divide en k particiones manteniendo la proporción de clases, y se entrena k veces dejando cada vez una partición fuera para test. En este trabajo se ha empleado $k = 5$.

Sin embargo, la validación cruzada estratificada no diagnostica el sesgo de dominio. En el problema que intentamos resolver, los vídeos provienen de distintas plataformas (Instagram, X/Twitter, grabación de móvil, descargas), cada una con un perfil de compresión característico. Si el modelo aprende a diferenciar plataformas en lugar de diferenciar un vídeo real de uno sintético, puede obtener un AUC excelente en validación cruzada y, sin embargo, fallar de forma sistemática al enfrentarse a una plataforma nueva.

Para confirmar y diagnosticar este problema, se ha recurrido a una estrategia variante de Leave-One-Out Cross-Validation (LOOCV); la estrategia Leave-One-Platform-Out (LOPO). Esta consiste en entrenar el modelo con todas las plataformas excepto una y evaluarlo exclusivamente sobre la plataforma omitida. Este procedimiento se repite sobre cada plataforma y proporciona una evaluación honesta sobre la capacidad de diferenciación entre dominios. La diferencia entre el AUC obtenido de la validación cruzada estratificada y el de LOPO definirá la magnitud del sesgo de dominio. Cuando esta diferencia supera ciertos valores (en este caso se ha fijado en 0.1), el modelo unificado deja de ser una solución apropiada.

2.4 Modelos multimodales

Las técnicas forenses descritas hasta aquí operan en el dominio de la señal: trabajan sobre propiedades estadísticas de los píxeles, sobre estructuras de compresión y sobre metadatos. Existen, sin embargo, situaciones en las que una persona detecta inmediatamente que un vídeo es sintético sin necesidad de medir nada: un personaje con seis dedos, un texto ilegible en un cartel del fondo, una sombra que no se corresponde con la fuente de luz visible, o una escena físicamente imposible. Estas son las señales semánticas o de contexto, y constituyen un canal complementario al análisis forense de bajo nivel.

Para incorporar este nivel de análisis, este trabajo recurre a un modelo multimodal de lenguaje y visión (Vision-Language Model, VLM). Los VLM son sistemas capaces de procesar conjuntamente imagen y texto, generando descripciones, respondiendo preguntas sobre imágenes o razonando sobre contenido visual en lenguaje natural.

En este trabajo he empleado Gemini 2.5 Flash de Google, accedido mediante su API pública. El modelo recibe cuatro *frames* representativos del vídeo (uno inicial, dos intermedios y uno final) junto con un *prompt* estructurado que le pide evaluar la coherencia física de la escena, la presencia de artefactos típicos de IA y la verosimilitud general. La respuesta se parsea en cinco campos (contexto, coherencia, artefactos, veredicto, confianza, razón) que se integran en el informe final.

La elección de un modelo flash responde a un equilibrio entre coste (gratis), latencia y capacidad de razonamiento adecuado para la tarea, que en este caso trabaja como función complementaria al análisis global.

2.5 Explicabilidad: SHAP

Un modelo de aprendizaje automático que clasifica un vídeo como sintético tiene poco valor operativo si no es capaz de justificar por qué. En contextos forenses, periodísticos o judiciales, la explicabilidad de la decisión es tan importante como su corrección. Esto es especialmente cierto cuando el destinatario del veredicto no es un experto técnico.

El marco teórico de la explicabilidad moderna en aprendizaje automático parte de un concepto procedente de la teoría de juegos cooperativos: los valores de Shapley [29]. Dado un "juego" en el que varios jugadores cooperan para obtener un pago total, los valores de Shapley distribuyen ese pago de forma justa entre los jugadores en función de su contribución marginal. Lundberg y Lee [30] trasladaron este concepto al aprendizaje automático con la propuesta de SHAP (SHapley Additive exPlanations): para una predicción concreta, el valor SHAP de cada *feature* mide su contribución marginal a esa predicción respecto a la predicción media del modelo.

Sobre modelos basados en árboles, SHAP cuenta con una implementación eficiente, TreeSHAP [31], que reduce la complejidad computacional de exponencial a polinómica. Esto es lo que permite, en este trabajo, calcular en pocos segundos la atribución de cada *feature* a cada predicción individual.

La integración de SHAP en este sistema cumple dos funciones. Por un lado, explica al usuario final a través de la web qué señales han llevado a clasificar un vídeo como auténtico o sintético, ofreciendo una explicación entendible para el usuario. Por otro, sirve de apoyo para el desarrollo como una herramienta de depuración extra: si SHAP revela que el modelo se apoya desproporcionadamente en una métrica sospechosa de capturar sesgo de plataforma, es un indicio de que el clasificador no está aprendiendo lo que pretendíamos.

Capítulo 3. Metodología y material empleado

3.1 Metodología

La metodología de este proyecto va de la mano con la situación del problema al que nos enfrentamos. La detección de vídeo sintético no tiene una solución cerrada, por lo que cada decisión técnica aporta información nueva que condiciona las siguientes, lo que lleva a adaptar el plan a medida que se van realizando avances.

Por todo esto, se ha optado por un plan basado en sprints, cada uno con objetivos concretos definidos al principio de la semana. Estos ciclos de trabajo cortos permiten incorporar o adaptar soluciones y procesos rápidamente: por ejemplo, los resultados de la validación LOPO revelaron un sesgo de dominio importante, pero el plan de trabajo permitió reorganizar la planificación de los siguientes esprints hacia el desarrollo de modelos especializados por plataforma sin desestructurar el plan general.

3.1.1 Fases del proyecto

El trabajo ha sido desarrollado a lo largo de cuatro grandes fases, cada uno compuesta por varios sprints:

Fase 1 – Análisis del problema y construcción del *dataset*. Esta primera fase abarca desde la investigación y revisión del estado del arte en detección de vídeo sintético, la identificación de métricas y trazas forenses relevantes y la construcción manual del *dataset*. Este *dataset* se ha estructurado en dos clases (*real/fake*) y, dentro de cada clase, los vídeos se han etiquetado en subcarpetas dependiendo de su plataforma de origen (Instagram, Twiteer y Descarga/Grabación). La descarga de vídeos procedentes de redes sociales se ha automatizado mediante yt-dlp, que genera adicionalmente un archivo .info.json con metadatos de origen que se aprovechan en las siguientes fases.

Fase 2 – Diseño e implementación del pipeline forense. Esta fase ha cubierto el desarrollo modular de la capa de extracción de *features*: análisis de metadatos con ExifTool, extracción de *frames* con FFmpeg, análisis en el dominio espacial (nitidez, FFT, DCT), análisis temporal (flujo óptico mediante Farnebäck, estructura IPB), análisis de ruido (Shapiro-Wilk, momentos estadísticos, correlación inter-canal) y análisis fotométrico. Una optimización relevante introducida en esta fase fue la lectura única de *frames* en memoria, que sustituyó al esquema inicial de lectura repetida desde disco para cada análisis y redujo el tiempo de procesado.

Fase 3 – Entrenamiento, evaluación y optimización de modelos. En esta fase se comparan los cuatro algoritmos descritos en el Capítulo 2 (Regresión Logística, Random Forest, Gradient Boosting, XGBoost), se ejecuta la validación cruzada estratificada para selección de modelo y la validación LOPO para diagnóstico de sesgo de dominio, se entrenan los modelos especializados por plataforma, se desarrolla el enrutador y se integra la capa multimodal con Gemini junto con la explicación mediante SHAP. También se organiza el *holdout* de evaluación para generar estadísticas y gráficas de rendimiento y fiabilidad del sistema.

Fase 4 – Aplicación web y documentación. La última fase ha cubierto la construcción del servidor Flask, el diseño del *frontend* con HTML/CSS/JavaScript, la integración del streaming de eventos mediante Server-Sent Events (SSE) para informar al usuario del progreso en tiempo real, y la redacción de la memoria.

3.2 Material empleado

3.2.1 Lenguaje de programación y librerías

El lenguaje empleado en todo el desarrollo del proyecto ha sido Python por su adaptabilidad para cumplir las funciones planteadas y la disponibilidad de las librerías necesarias, las cuales son:

- **Python** [32] es un lenguaje de programación interpretado, de alto nivel y tipado dinámico, consolidado como estándar en muchos ámbitos como la inteligencia artificial, el análisis de datos o la visión por computador. Cuenta con un amplio catálogo de librerías y mucha documentación accesible.
- **Numpy** [33] es la librería principal y fundamental para el cálculo numérico en Python, proporciona arrays multidimensionales y operaciones vectorizadas. Es de vital importancia para el proyecto, pues todas las métricas forenses se calculan como operaciones estadísticas sobre arrays de Numpy.
- **SciPy** [34] amplía las capacidades de Numpy con funciones estadísticas avanzadas, como momentos estadísticos de orden superior del ruido residual (asimetría y curtosis) o para el test de Shapiro-Wilk, presentes en el trabajo.
- **Pandas** [35] es la librería referencia para la manipulación de datos tabulares organizados en estructuras de tipo *DataFrame*. Sirve para la gestión del *dataset* y la lectura y escritura de los archivos CSV utilizados para los entrenamientos.
- **Scikit-learn** es la librería utilizada para el aprendizaje automático clásico e integra implementaciones de algoritmos de clasificación, esquemas de validación y métricas de evaluación. Lo utilizo para la comparación de clasificadores (Random Forest, Gradient Boosting y Regresión Logística), los esquemas de validación cruzada (Stratified K Fold y LOPO) y las métricas de evaluación (ROC-AUC, F1, Recall y matrices de confusión).
- **XGBoost** es una implementación optimizada de Gradient Boosting que introduce tratamiento eficiente de valores ausentes y paralelización. Se utiliza como cuarto clasificador, comparada con las alternativas de scikit-learn.
- **SHAP** es una librería de explicabilidad basada en los valores de Shapley de la teoría de juegos cooperativos, atribuyendo a cada *feature* su contribución de una predicción concreta. Se ha empleado a través de su implementación para modelos basados en árboles, **TreeSHAP**, que reduce la complejidad y permite calcular las contribuciones en pocos segundos, lo que lo hace óptimo y eficiente.
- **Matplotlib** [36] es la librería base para la generación de gráficas. En este caso, la he utilizado para la generación de las figuras de evaluación presentes en el documento y poder interpretar los resultados.

3.2.2 Procesado de vídeo e imagen

- **OpenCV** [37] es la librería de visión por computador más extendida y proporciona implementaciones eficientes de muchas operaciones sobre imagen y vídeo. En ella recae el peso de la mayor parte de extracción de *features*: la lectura de *frames*, la conversión entre espacios de color, el operador laplaciano para la nitidez, la transformada de Fourier o el flujo óptico, la aberración cromática y las correlaciones inter-canal.
- **FFmpeg** [38] es la referencia para la manipulación de contenido multimedia. En este proyecto actúa como motor de extracción de *frames* a una tasa fija de 5 fps y con un redimensionado a un ancho de 640 píxeles, normalización fundamental para que las métricas extraídas sean comparables entre diferentes vídeos.

- **FFprobe** es parte del paquete FFmpeg y permite inspeccionar la estructura interna de un archivo multimedia. Lo he empleado para el análisis de la estructura IPB de los vídeos, obteniendo el tipo de cada *frame*, la secuencia del GOP y el tamaño relativo entre los *frames* de tipo I y P. Esta información es la base del bloque de métricas de compresión.
- **ExifTool** [39] es una herramienta de extracción de metadatos con soporte para más de cien formatos de archivo y miles de etiquetas.
- **MediaPipe** [40] es un conjunto de soluciones de Google para el procesamiento de medios en tiempo real, entre los cuales se incluye la detección de puntos referencia faciales para construir la base del análisis facial. Detecta 478 puntos de la cara sobre los que calcular parámetros fisiológicos y geométricos.

3.2.3 API externa

El sistema integra un servicio externo de razonamiento multimodal mediante el SDK de Google Generative AI, que da acceso a los modelos de la familia Gemini a través de peticiones HTTP. Estos modelos son capaces de procesar conjuntamente imagen y texto, generando respuestas en lenguaje natural a partir del *prompt* recibido.

He optado por el modelo Gemini 2.5 Flash [41], a través del cual accedo con una clave API obtenida de Google AI Studio. En grandes desarrollos se pueden contratar servicios superiores con muchas más peticiones para sostener un uso frecuente. La elección de una variante flash se debe a su coste gratuito y su funcionalidad más que suficiente para el proyecto desarrollado, conjuntamente con su latencia y capacidad de razonamiento suficientes para la tarea.

3.2.4 Aplicación web

Para la capa de interfaz y presentación se han utilizado:

- **Flask** [42], un *microframework* de servidor web para Python, cuyas principales fortalezas son su ligereza e imponer un número mínimo de decisiones de arquitectura. Lo he elegido frente a alternativas más comunes como Django o FastAPI porque son más pesadas y la aplicación gira en torno a un *endpoint* principal de análisis y no requiere de extras como autenticación, paneles de administración ni modelos de datos complejos. Es decir, aporta simplicidad para exponer el pipeline forense sin introducir dependencias innecesarias.
- **HTML, CSS3 y JavaScript** [43] [44] [45]: La interfaz se ha construido con las tecnologías web estándar aprendidas a lo largo de la carrera. Con HTML puedes realizar la estructura base de los componentes de la web. Con CSS desarrollo el estilo y la distribución visual que le quiero dar a la página y asegurar que funcione en diferentes dispositivos, como el ordenador y el móvil. JavaScript es el lenguaje que le da a la aplicación la capa interactiva, lo que permite gestionar eventos de actualización en la interfaz e interacciones con el servidor.
- **Server-Sent Events o SSE** [46] es una tecnología que permite el envío unidireccional de eventos del servidor al cliente a través de una conexión HTTP persistente. En mi caso lo utilizo para transmitir el avance del análisis en tiempo real dividido por fases. Planteé la alternativa de WebSockets pero era innecesaria porque la comunicación requerida era unidireccional, por lo que opté por un modelo más sencillo al igual que hice con Flask.
- **Yt-dlp** [47] es una herramienta para la descarga de contenido multimedia con soporte para cientos de sitios web. Su uso se fundamenta principalmente en la necesidad de descargar videos de redes sociales que no te permiten descargarlos o que requieren una cuenta premium para ello. Lo he utilizado tanto para la construcción del *dataset* como

para la descarga de los vídeos a analizar en la aplicación a través de URL y la generación del archivo .info.json de los vídeos, que aporta información de mucho valor.

3.2.5 *Dataset*

El *dataset* propio construido para este trabajo se compone de más de 300 vídeos, distribuidos aproximadamente al 50% entre reales y *fakes*. Las fuentes empleadas para la recolección han sido:

- **Grabaciones** de vídeos propios desde dispositivos móviles, **descargas** de cuentas verificadas de redes sociales, archivos personales y vídeos de stock de dominio público.
- **Vídeos sintéticos**: generaciones propias mediante webs públicas o de demostración como Sora, Veo, Runway, Kling u otros modelos generativos accesibles junto con galerías de arte donde la gente publica sus propias creaciones generadas. Para los vídeos de redes sociales navegaba por ellas y guardaba vídeos a medida que me iban saliendo para que el algoritmo me mostrara más y guardaba las cuentas para encontrar similares.

Todos los vídeos descargados desde redes sociales se han conservado junto con su archivo de .info.json de yt-dlp, lo que permite el rastreo completo del origen y la etiquetación automática por plataforma.

Capítulo 4. Desarrollo del proyecto

4.1 Funcionamiento general

El sistema construido en este trabajo es un pipeline completo para la detección de vídeo por inteligencia artificial. Respecto a la arquitectura, en lugar de delegar todo el problema en una red neuronal que estudia los píxeles y da un veredicto, se ha optado por un esquema modular en el que cada capa aporta un punto de vista diferente del problema y, sobretodo, una visión interpretable.

El flujo del sistema sigue los siguientes pasos:

1. Recibe el vídeo, ya sea en forma de archivo o como una URL de una red social.
2. Detecta el origen en base al archivo JSON o a lo que el usuario marque en la aplicación.
3. Selección por parte del usuario del segmento a analizar, eligiendo entre 30 o 60 segundos, el vídeo completo o un fragmento concreto (por ejemplo, del segundo 15 al 25).
4. Extrae los *frames* a 5 fps con FFMpeg, redimensionados a una resolución máxima de 640p, cargados una única vez en memoria.
5. Extracción de los *features* forenses sobre los *frames* y los metadatos.
6. Clasificación con el modelo apropiado en base al origen detectado (modelos especializados o unificado).
7. Cálculo de la explicación SHAP sobre la predicción.
8. Análisis de contexto con Gemini 2.5 Flash sobre cuatro *frames* representativos.
9. Genera el informe final combinando todas las capas y entrega el resultado al usuario.

Figura 6. Interfaz página principal

4.2 Construcción del *dataset*

Parte de la precisión del modelo final está condicionada por la calidad del *dataset* sobre el que se entrena. Primero busqué *datasets* públicos extensos, pero estos estaban obsoletos respecto a la calidad de los generadores actuales, por lo que los descarté ya que no ayudarían a enfrentar el problema de manera óptima. Además, tampoco estaban etiquetados y clasificados para dividirlos por su procedencia, lo que es fundamental para el análisis de sesgo de dominio que se aborda en este trabajo.

4.2.1 Estructura y etiquetado

El *dataset* se organiza en dos carpetas raíz, *datasets/real/* y *datasets/fake/*, que se recorren automáticamente por el script `extraer_datos.py`. Cada archivo de vídeo se acompaña, cuando proviene de una red social, de su correspondiente `.info.json` generado por `yt-dlp`, del que se extrae el campo extractor para etiquetar la plataforma. La función `leer_origen_ydlp` parsea este campo y lo mapea a una de las cuatro categorías de origen consideradas: `instagram`, `twitter`, `grabación/descarga` (grabaciones sin `.info.json`, asumidas como captura directa o descarga de galería de internet o similares) y `desconocido` (extractor presente pero no identificado).

4.2.2 Fuentes

Comentadas en el subapartado 3.5.2

4.2.3 Extracción de *features* para entrenamiento

El script `extraer_datos4.py` automatiza el procesamiento del *dataset* completo. Para cada vídeo, lee la lista de *frames* del disco una única vez y reutiliza esa lista en todos los análisis posteriores, lo que reduce de forma significativa el coste de entrada/salida respecto a una implementación que volviera a leer los *frames* para cada métrica. El procesamiento se paraleliza mediante un `ProcessPoolExecutor` de la librería `concurrent.futures`, escalando con el número de núcleos

disponibles. El resultado es un archivo *features_v2.csv* que será la entrada que recibirá el código de entrenamiento.

4.3 Extracción de *features* forenses

El pipeline forense extrae 30 métricas numéricas, que forman un total de 8 bloques temáticos o conceptuales. A ellas se le añaden las variables de origen para clasificarlas en el entrenamiento.

Ocultar métricas

META_NUM_ALERTAS 2	IPB_PCT_I 0.4132	IPB_PCT_P 99.5868	IPB_PCT_B 0
IPB_GOP_MEDIO 242	IPB_GOP_STD 0	IPB_RATIO_TAMANO_IP 1.9688	IPB_NUM_ALERTAS 2
NITIDEZ_MEDIA 62.8397	NITIDEZ_STD 6.2220	NITIDEZ_OUTLIER_PCT 0.0200	FFT_ENERGIA_MEDIA 144.3227
FFT_RATIO_ALTA_BAJA 23.1590	FFT_RATIO_STD 0.1615	FLUJO_MEDIO 5.2883	FLUJO_STD 0.8866
FLUJO_SALTO_MAXIMO 1.7437	FLUJO_NUM_GLITCHES 2	LUZ_STD 1.3390	BRILLO_STD 11.8977
BRILLO_TENDENCIA 0.6937	CA_DESPASE_MEDIO 0.0629	CA_DESPASE_STD 0.0366	DCT_RATIO 2.0400
RUIDO_SHAPIRO_P 0	RUIDO_STD 2.9270	RUIDO_ASINETRIA 2.1587	RUIDO_CURTOSIS 49.9621
CORR_RG 0.9206	CORR_RB 0.7542	CORR_GB 0.9134	NL_PROB_IA 0.7668

Nuevo análisis

Figura 7. *Features* medidas de un vídeo

4.3.1 Análisis de nitidez

Para cada *frame* se calcula la varianza del operador laplaciano, una métrica clásica de nitidez basada en la cantidad de bordes presentes en la imagen. De la serie temporal resultante se extraen tres *features*: la nitidez media, su desviación típica y el porcentaje de *frames* considerados atípicos en términos de nitidez, definidos como aquellos cuya nitidez se desvía más de dos desviaciones típicas de la media. Un vídeo generado por IA tiende, o bien a una nitidez excesivamente regular (modelo conservador), o bien a presentar saltos abruptos en *frames* concretos (modelo inestable).

```
def calc_nitidez(frames):
    if not frames:
        return 0.0, 0.0, 0.0
    vals = np.array([cv2.Laplacian(f, cv2.CV_64F).var() for f in frames])
    mu, sg = float(vals.mean()), float(vals.std())
    outliers = float(np.mean(np.abs(vals - mu) > 2 * sg))
    return mu, sg, outliers
```

Figura 8. Cálculo de parámetros de nitidez

4.3.2 Análisis en el dominio de la frecuencia (FFT)

Cada *frame* se transforma al dominio de la frecuencia mediante FFT 2D y se aplica un *fftshift* para situar la componente DC en el centro de la imagen. Sobre el espectro de magnitud se define una máscara circular que separa las bajas frecuencias (un disco central cuyo radio es 1/8 de la dimensión menor del *frame*) de las altas frecuencias (el complementario). De esta separación

surgen las *features*: la energía media del espectro, el ratio entre energía de altas y bajas frecuencias y la variación temporal de ese ratio entre *frames*.

```
def calc_fft(frames):
    if not frames:
        return 0.0, 0.0, 0.0
    h, w = frames[0].shape
    mascara = np.zeros((h, w))
    cv2.circle(mascara, (w // 2, h // 2), min(h, w) // 8, 1, -1)
    inv = 1 - mascara
    energias, ratios = [], []
    for img in frames:
        mag = 20 * np.log(np.abs(np.fft.fftshift(np.fft.fft2(img))) + 1)
        energias.append(mag.mean())
        ratios.append((mag * inv).sum() / ((mag * mascara).sum() + 1e-6))
    return float(np.mean(energias)), float(np.mean(ratios)), float(np.std(ratios))
```

Figura 9. Cálculo FFT

4.3.3 Flujo óptico

El movimiento entre *frames* consecutivos se estima con el algoritmo de Farnebäck , con los parámetros estándar de OpenCV: pirámide de tres niveles con factor de escala 0,5, ventana de quince píxeles, tres iteraciones, polinomio expandido de orden cinco y desviación gaussiana 1,2. La magnitud del flujo se obtiene con `cv2.cartToPolar` y de la serie temporal se extraen el movimiento medio, su desviación típica, el salto máximo entre *frames* consecutivos y un contador de glitches, definidos como saltos que superan en más de tres desviaciones típicas la media móvil de los saltos anteriores.

```
def calc_flujo_optico(frames):
    if len(frames) < 2:
        return 0.0, 0.0, 0.0, 0
    h, w = frames[0].shape
    movs, saltos, glitches = [], [], 0
    prev = frames[0]
    for curr in frames[1:]:
        if curr.shape != prev.shape:
            curr = cv2.resize(curr, (w, h))
        flujo = cv2.calcOpticalFlowFarneback(prev, curr, None, 0.5, 3, 15, 3, 5, 1.2, 0) # parámetros por defecto
        mag, _ = cv2.cartToPolar(flujo[...], 0, flujo[...], 1)
        movs.append(float(mag.mean()))
        if len(movs) > 1:
            d = abs(movs[-1] - movs[-2])
            saltos.append(d)
            if len(saltos) > 5:
                arr = np.array(saltos[:-1])
                if d > arr.mean() + 3 * arr.std():
                    glitches += 1
        prev = curr
    return (float(np.mean(movs)), float(np.std(movs)),
            float(max(saltos)) if saltos else 0.0, glitches)
```

Figura 10. Cálculo del flujo óptico

4.3.4 Análisis fotométrico

La dirección estimada de iluminación se obtiene aplicando los operadores Sobel sobre una versión suavizada del *frame* y calculando el arcotangente del gradiente. De la serie temporal de direcciones se obtiene su desviación típica. Adicionalmente se calcula la desviación típica del brillo medio entre *frames* y la tendencia lineal del brillo a lo largo del vídeo, ajustando un polinomio de primer orden mediante `np.polyfit`. Una pendiente significativa indica un fade artificial inducido por el generador.

```
def calc_fotometria(frames):
    if not frames:
        return 0.0, 0.0, 0.0
    direcciones, brillos = [], []
    for img in frames:
        b1 = cv2.GaussianBlur(img, (15, 15), 0)
        sx = cv2.Sobel(b1, cv2.CV_64F, 1, 0, ksize=5)
        sy = cv2.Sobel(b1, cv2.CV_64F, 0, 1, ksize=5)
        direcciones.append(float(np.arctan2(sy, sx).mean() * 180 / np.pi))
        brillos.append(float(img.mean()))
    pendiente = float(np.polyfit(range(len(brillos)), brillos, 1)[0])
    return float(np.std(direcciones)), float(np.std(brillos)), pendiente
```

Figura 11. Análisis fotométrico

4.3.5 Aberración cromática

Sobre el primer *frame* en color, se extraen cinco parches de 100×100 píxeles (centro y cuatro esquinas) en los canales rojo y verde. Para cada par se calcula la correlación de fase, que devuelve el desplazamiento sub-píxel que mejor alinea ambos parches. La media de los desplazamientos y su desviación típica entre zonas constituyen las dos *features* de este bloque. Una lente óptica real produce mayor aberración en los bordes que en el centro, por lo que una desviación muy baja —aberración perfectamente uniforme entre el centro y los bordes— delata una "lente sintética" que no respeta la física de la óptica.

```
def _parche(canal, y, x, t=100): # evitar errores de cálculo en bordes
    p = canal[y - t // 2:y + t // 2, x - t // 2:x + t // 2]
    return p if p.shape == (t, t) else None

def calc_aberracion_cromatica(color):
    if color is None:
        return 0.0, 0.0
    g, r = cv2.split(color.astype(np.float32))
    h, w = color.shape[:2]
    t = 100
    desps = []
    for y, x in [(h//2, w//2), (t, t), (t, w-t), (h-t, t), (h-t, w-t)]:
        pg, pr = _parche(g, y, x), _parche(r, y, x)
        if pg is None or pr is None or pg.sum() == 0:
            continue
        shift, _ = cv2.phaseCorrelate(pg, pr)
        desps.append(float(np.hypot(*shift)))
    if not desps:
        return 0.0, 0.0
    return float(np.mean(desps)), float(np.std(desps))
```

Figura 12. Cálculos de aberración cromática

4.3.6 Análisis DCT y patrones de compresión

El módulo de compresión analiza la presencia de discontinuidades en los bordes de los bloques de 8×8 y 16×16 píxeles, característicos de los códecs basados en DCT. Calcula los gradientes horizontal y vertical del *frame* y cuenta cuántos bordes de bloque presentan un gradiente que supera en un 20 % la media. La ratio entre bordes de 8×8 y de 16×16 cuantifica el grado de recompresión: un vídeo que ha pasado por varias recodificaciones presenta una ratio elevada.

```
def calc_dct(frames):
    if not frames:
        return 0.0
    img = frames[0]
    gx = np.abs(np.diff(img.astype(float), axis=1))
    gy = np.abs(np.diff(img.astype(float), axis=0))
    mgx, mgy = gx.mean(), gy.mean()

    def bloques(bs):
        h, w = img.shape
        c = sum(1 for i in range(bs, h, bs) if i < gy.shape[0] and gy[i].mean() > mgy * 1.2)
        c += sum(1 for j in range(bs, w, bs) if j < gx.shape[1] and gx[:, j].mean() > mgx * 1.2)
        return c

    return float(bloques(8) / (bloques(16) + 1))
```

Figura 13. Cálculos bloques DCT

4.3.7 Análisis estadístico del ruido

Se extrae el ruido residual del primer *frame* por sustracción de su versión filtrada con un filtro de mediana (cv2.medianBlur con kernel 5×5). Sobre el residuo se calculan:

- Test de Shapiro-Wilk sobre una muestra aleatoria de 5 000 píxeles, devolviendo el p-valor de la hipótesis nula de gaussianidad. Un p-valor alto indica un ruido "demasiado perfecto" para ser real.
- Desviación típica del ruido.
- Asimetría o skewness.
- Curtosis Correlación intercanal RGB.

```
def calc_ruido(frames):
    if not frames:
        return 0.0, 0.0, 0.0, 0.0
    img = frames[0]
    ruido = img.astype(float) - cv2.medianBlur(img, 5).astype(float)
    flat = ruido.flatten()
    muestra = np.random.choice(flat, 5000, replace=False) if len(flat) > 5000 else flat
    p = shapiro(muestra)
    return float(p), float(ruido.std()), float(stats.skew(flat)), float(stats.kurtosis(flat))
```

Figura 14. Análisis del ruido residual

4.3.8 Correlación intercanal RGB

Se calculan las correlaciones de Pearson entre los canales del primer *frame* en color: rojo-verde, rojo-azul y verde-azul. Correlaciones simultáneamente superiores a 0,95 en RG y RB indican canales casi idénticos, lo cual sugiere generación sin variación cromática real o monocromatismo artificial.

```
def calc_correlacion_rgb(color):
    if color is None:
        return 0.0, 0.0, 0.0
    b, g, r = cv2.split(color)
    rf, gf, bf = r.flatten(), g.flatten(), b.flatten()
    rg = float(np.corrcoef(rf, gf)[0, 1])
    rb = float(np.corrcoef(rf, bf)[0, 1])
    gb = float(np.corrcoef(gf, bf)[0, 1])
    return [rg, rb, gb]
```

Figura 15. Correlaciones canales de color

4.3.9 Estructura IPB

Mediante llamada a `ffprobe` se obtiene la lista de tipos de *frame* del vídeo (I, P, B) y sus tamaños. De ahí se obtiene el porcentaje presente de cada tipo, el tamaño medio y la desviación típica del GOP, el ratio entre el tamaño media de un *I-frame* y un *P-frame* y un contador de alertas que aumenta en base a diferentes anomalías: ausencia de *B-frames*, exceso de *I-frames*, GOP muy corto, cadencia GOP excesivamente estable, y ratio I/P bajo (es decir, como los *I-frames* ocupan mucho más en memoria, por lo que la alerta saltaría cuando los *I-frames* no son mucho más grandes que los *P-frames*).

```
def calc_ipb(ruta_video):
    cmd = ['ffprobe', '-v', 'quiet', '-select_streams', 'v:0', '-show_frames',
          '-show_entries', 'frame=pict_type,pkt_size', '-of', 'json', ruta_video]
    res = subprocess.run(cmd, capture_output=True, text=True)
    frames = json.loads(res.stdout).get('frames', []) if res.stdout.strip() else []
    if not frames:
        return 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0

    conteos = {'I': 0, 'P': 0, 'B': 0}
    tamanos = {'I': [], 'P': [], 'B': []}
    gops, gop = [], 0
    for fr in frames:
        t, sz = fr.get('pict_type', '?'), int(fr.get('pkt_size', 0))
        if t in conteos:
            conteos[t] += 1
            tamanos[t].append(sz)
        if t == 'I':
            if gop > 0:
                gops.append(gop)
            gop = 1
        else:
            gop += 1
    if gop > 0:
        gops.append(gop)

    total = sum(conteos.values()) or 1
    pct = {k: conteos[k] / total * 100 for k in 'IPB'}
    gm = float(np.mean(gops)) if gops else 0.0
    gs = float(np.std(gops)) if gops else 0.0
    mi = np.mean(tamanos['I']) if tamanos['I'] else 0
    mp = np.mean(tamanos['P']) if tamanos['P'] else 1
    rip = float(mi / (mp + 1e-6))

    alertas = sum([conteos['B'] == 0, pct['I'] > 20,
                  gs < 1.0 and gm > 0, rip < 1.5])
    return pct['I'], pct['P'], pct['B'], gm, gs, rip, alertas
```

Figura 16. Cálculos estructura IPB

4.3.10 Metadatos

ExifTool permite obtener con subprocessos un JSON con las etiquetas presentes en el contenedor de vídeo. Se tienen en cuenta diferentes condiciones, como incoherencias temporales (un archivo con fecha de creación posterior a fecha de modificación), un fabricante sospechoso (o cualquier fabricante que contenga “ai”) y ausencia de varios campos presentes en una grabación real.

```
def calc_metadatos(ruta_video):
    res = subprocess.run([RUTA_EXIFTOOL, '-j', ruta_video], capture_output=True, text=True)
    if not res.stdout.strip():
        return 3
    meta = json.loads(res.stdout)[0]
    alertas = 0
    fab = meta.get('Make', '').lower()
    if fab in ['unknown', 'synthesia', 'runway', 'generic', 'seedance', 'sora', 'openai', 'gemini', 'kling'] or 'ai' in fab:
        alertas += 1
    if sum(1 for c in ('Make', 'Model', 'CreateDate', 'Software') if not meta.get(c)) >= 3:
        alertas += 1
    fc, fm = meta.get('CreateDate'), meta.get('ModifyDate')
    if fc and fm and fc > fm:
        alertas += 1
    return alertas
```

Figura 17. Lectura de metadatos

4.3.11 Análisis facial

Además de las métricas forenses anteriores, mediante MediaPipe se obtienen y evalúan una serie de parámetros como la frecuencia y simetría del parpadeo, la variación de la apertura bucal, micro temblores de los landmarks, variabilidad del EAR (Eye Aspect Ratio), simetría facial, estabilidad de los bordes y consistencia de iluminación entre el rostro y el fondo.

Esta rama del trabajo replica la estructura del análisis principal: se extraen los datos faciales a partir de los *frames*, se construye un *dataset* independiente y se entrena un modelo facial propio comparando algoritmos y haciendo validación cruzada. Cuando hay un rostro en más del 50% de los *frames* se activa el análisis y se muestra el veredicto independiente. Si no se detecta un rostro bajo las condiciones impuestas, el módulo devuelve que no se han encontrado caras y no se muestra nada en la interfaz.

4.4 Entrenamiento del modelo unificado

Una vez se han extraído los valores del *dataset* completo, realizamos un entrenamiento, junto con su evaluación y la elección del clasificador óptimo.

4.4.1 Comparación de clasificadores

Antes de llegar al clasificador, todos los datos pasan por una función, *LimpiadorFeatures*, que es un transformador integrado en el *pipelin*. Consiste en dos operaciones sucesivas: primero sustituye los valores ausentes porque haya sucedido cualquier fallo por la mediana de cada descriptor (un ejemplo sería FFprobe, que podría no devolver correctamente el GOP); después recorta los valores que estén muy por encima de la media de las desviaciones típicas de la media, con el objetivo de no condicionar la predicción.

Lo implemento durante el entrenamiento y no preproceso los datos antes para que en cada tanda de la validación cruzada aprenda sus estadísticas (mediana, media, desviación típica) únicamente sobre los datos de ese entrenamiento en esa tanda y los aplique al test sin haberlos visto. Si se calcularan sobre todos los vídeos antes de la validación, los valores contaminarían el entrenamiento y las métricas podrían ser más optimistas de lo que deberían.

En total hay cuatro pipelines en el entrenamiento, uno para cada algoritmo:

- **Random Forest:** 300 árboles, `min_samples_leaf=2`, `class_weight='balanced'`.

- **Gradient Boosting:** 200 estimadores, learning rate 0,05, profundidad máxima 4, subsample 0,8.
- **Regresión Logística:** con StandardScaler previo, C=1,0, class_weight='balanced', 1000 iteraciones.
- **XGBoost:** 300 estimadores, learning rate 0,05, profundidad máxima 5, subsample y colsample_bytree ambos a 0,8.

4.4.2 Evaluación: CV estratificada y LOPO

Como ya se ha mencionado anteriormente, la selección del modelo se hace mediante validación cruzada estratificada de 5 folds con la métrica ROC-AUC como criterio principal, pero teniendo en cuenta también Accuracy y F1-Score. Una vez decidido el mejor modelo, se realiza la evaluación Leave-One-Platform-Out: entrena con X-1 carpetas del *dataset* y evalúa sobre la omitida. La comparación entre el AUC de la CV estratificada y la media de los AUC por plataforma de LOPO nos dará lo que necesitamos saber para identificar un posible sesgo de dominio. Como la diferencia supera un 0.10, el sesgo de dominio es ya importante, lo que me ayudó a decidirme por optar a trabajar con modelos especializados por plataforma y manteniendo este modelo como modelo unificado de salvaguarda.

Finalmente, sobre una partición 80/20 de entrenamiento y test, se genera el reporte del entrenamiento con matriz de confusión, métricas por origen, e importancia de métricas.

4.4.3 Persistencia del modelo

El modelo final unificado se reentrena después de la evaluación LOPO sobre el *dataset* completo y se serializa en un único archivo que contiene los nombres de las métricas, los resultados de la CV, los del LOPO y el nombre del algoritmo de clasificación seleccionado.

4.5 Modelos especializados por plataforma

Después de conocer el sesgo de dominio entre plataformas, tomé la decisión de un esquema de modelos especializados por origen, lo que conlleva hacer entrenamientos individuales para cada plataforma.

Para este entrenamiento, el *dataset* completo se filtra por cada uno de los orígenes, generando un *dataset* independiente para poder entrenar cada modelo con el script que ya tenemos.

El resultado es un conjunto de archivos en formato modelo_(origen).pkl y un archivo JSON que registra la ruta de cada uno de los modelos, además de la ruta del modelo unificado del entrenamiento global.

4.6 Enrutamiento de modelos

Para poder dirigir el análisis hacia el modelo deseado, cargamos el JSON con los índices, comprueba si existe un modelo especializado para el origen detectado en el vídeo y devuelve esa ruta, si no, llama al modelo general. Lo que permite esto sería facilitar la incorporación de nuevas plataformas a este esquema de trabajo, ya que de esta manera solo habría que realizar su respectivo entrenamiento y añadir la ruta al índice.

4.7 Pipeline forense integrado

El módulo principal integra todas las capas anteriores en un solo flujo de trabajo integrado en una sola clase. Este se puede invocar en consola para realizar pruebas a lo largo del desarrollo y también se invoca en la aplicación web. El flujo es el siguiente:

1. Obtenemos la duración del vídeo y ofrecemos al usuario la posibilidad de elegir el fragmento o la cantidad de segundos a analizar.
2. En la selección de origen, marcamos de donde se ha obtenido el vídeo o elegimos que se detecte automáticamente.
3. Se invoca al enrutador de modelos para cargar el modelo correspondiente
4. Antes de extraer *frames*, se leen los metadatos con ExifTool y se evalúan las inconsistencias
5. Se realiza la extracción completa de *frames* con FFmpeg y se almacenan todos los *frames* en la memoria
6. Sucede la ejecución secuencial de todos los bloques de extracción de las *features* o métricas ya mencionadas.
7. Se construye el vector de *features* con el orden esperado por el modelo cargado y devuelve la probabilidad de IA del vídeo con la explicación SHAP.

4.8 Análisis de contexto con Gemini

El módulo de análisis de contexto en la aplicación implementa la capa de razonamiento semántico. Selecciona cuatro *frames* representativos del vídeo, situados aproximadamente al principio, al primer y segundo tercio y al final, los codifica y los envía a la API de Gemini 2.5 Flash junto con un *prompt* estructurado.

Este *prompt* es la pieza central del análisis multimodal y se ha diseñado para producir una respuesta parseable de forma robusta. Solicita al modelo que devuelva seis campos con una línea por campo: contexto, coherencia, artefactos, veredicto y justificación.

Como es una capa de apoyo, si la API da un problema o falla la comunicación, el sistema funcionará de igual manera sin el análisis de contexto, omitiendo esta sección.



Figura 18. Veredicto análisis de contexto en vídeo generado por IA

4.9 Explicación con SHAP

Tras obtener la predicción del modelo, el sistema calcula los valores SHAP de las *features* mediante TreeExplainer sobre el clasificador entrenado. Las contribuciones se ordenan por valor absoluto descendente y se seleccionan las diez más influyentes, descartando aquellas con magnitud inferior a 0,001.

Cada contribución se enriquece con tres elementos adicionales: el valor medido de esa *feature* en el vídeo analizado, el signo de la contribución (positiva empuja hacia el veredicto IA, negativa hacia el veredicto auténtico) y una descripción en lenguaje natural que explica al usuario, en términos accesibles, qué significa esa *feature* y por qué su valor concreto es indicio de uno u otro veredicto. Por ejemplo, para la *feature* ruido_shapiro_p, la descripción asociada es: "Test de gaussianidad del ruido. El ruido real de sensor no es perfectamente gaussiano. Si pasa el test con nota, el ruido es demasiado perfecto para ser real."

Esta combinación de SHAP junto con una descripción en lenguaje natural es lo que permite que un usuario sin formación técnica reciba no solo un veredicto, sino una justificación comprensible de ese veredicto.

En la siguiente figura podemos ver cómo sería el resultado del análisis de un vídeo generado por IA, con sus correspondientes explicaciones por *feature*:

4.10 Aplicación web

La aplicación web permite la interacción del sistema con el usuario final. El servidor está implementado en Flask y expone tres *endpoints* principales: subida de archivo, análisis desde URL y barra de progreso.

4.10.1 Backend

El servidor recibe el vídeo, lo guarda en un directorio temporal, genera un id único mediante UUID, y lanza el análisis en un hilo separado para no bloquear la respuesta HTTP. El progreso del análisis se publica en una cola y se entrega al cliente mediante Server-Sent Events (SSE). Cada paso del pipeline (extracción de *frames*, IPB, nitidez, FFT, flujo óptico, ...) emite un evento con un mensaje descriptivo, lo que permite a la interfaz mostrar una barra de progreso en tiempo real.

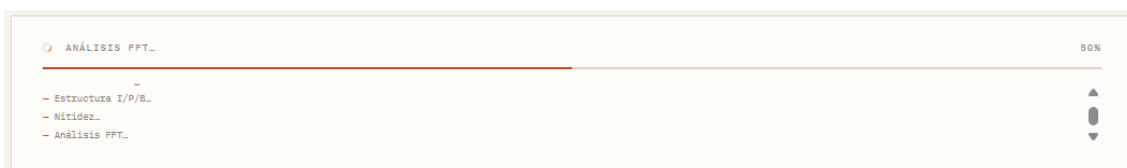


Figura 19. Barra de progreso

Para análisis desde URL, el *backend* llama a yt-dlp como subprocesso, descarga el vídeo en un directorio temporal con su *.info.json*, y continúa el procesado como en el caso de uso con archivo local.

Figura 20. Sección pegar URL para análisis y selección de fragmento de vídeo

La API entre el navegador y el servidor la forman seis *endpoints*: la interfaz principal, el *endpoint* de análisis que devuelve un identificador de tarea, el canal SSE para mostrar la barra de progreso en tiempo real, y tres *endpoints* de apoyo para entregar el vídeo, la miniatura y la duración.

4.10.2 Frontend

La interfaz, implementada con CSS y JavaScript vanilla, se organiza en torno a un *dropzone* central para subida de archivo y una pestaña alternativa para el análisis desde URL. Antes de iniciar el análisis, el usuario puede:

- Seleccionar manualmente el origen del vídeo (Instagram, Twitter, Grabación/Descarga, "no lo sé"), forzando el uso de un modelo especializado concreto, o dejar la opción "detectar automáticamente" para que el sistema decida.
- Elegir la duración del segmento a analizar o el fragmento concreto.

Durante el análisis, la barra de progreso se actualiza en tiempo real con los eventos SSE. Al finalizar, la interfaz muestra el veredicto principal (PROBABLEMENTE AUTÉNTICO /

SOSPECHOSO / SINTÉTICO según los umbrales 35 y 65 sobre la probabilidad ML), las evidencias forenses detectadas, la lista de las diez *features* más influyentes según SHAP con sus descripciones en lenguaje natural, y el análisis de contexto semántico. Si contiene un primer plano, muestra el veredicto facial.

El resultado completo se exporta también como JSON descargable para casos de uso forenses o periodísticos donde el informe deba archivarse.

Este sería el resultado de los tres análisis sobre un vídeo real procedente de Instagram, con su correspondiente explicación de las *features*:

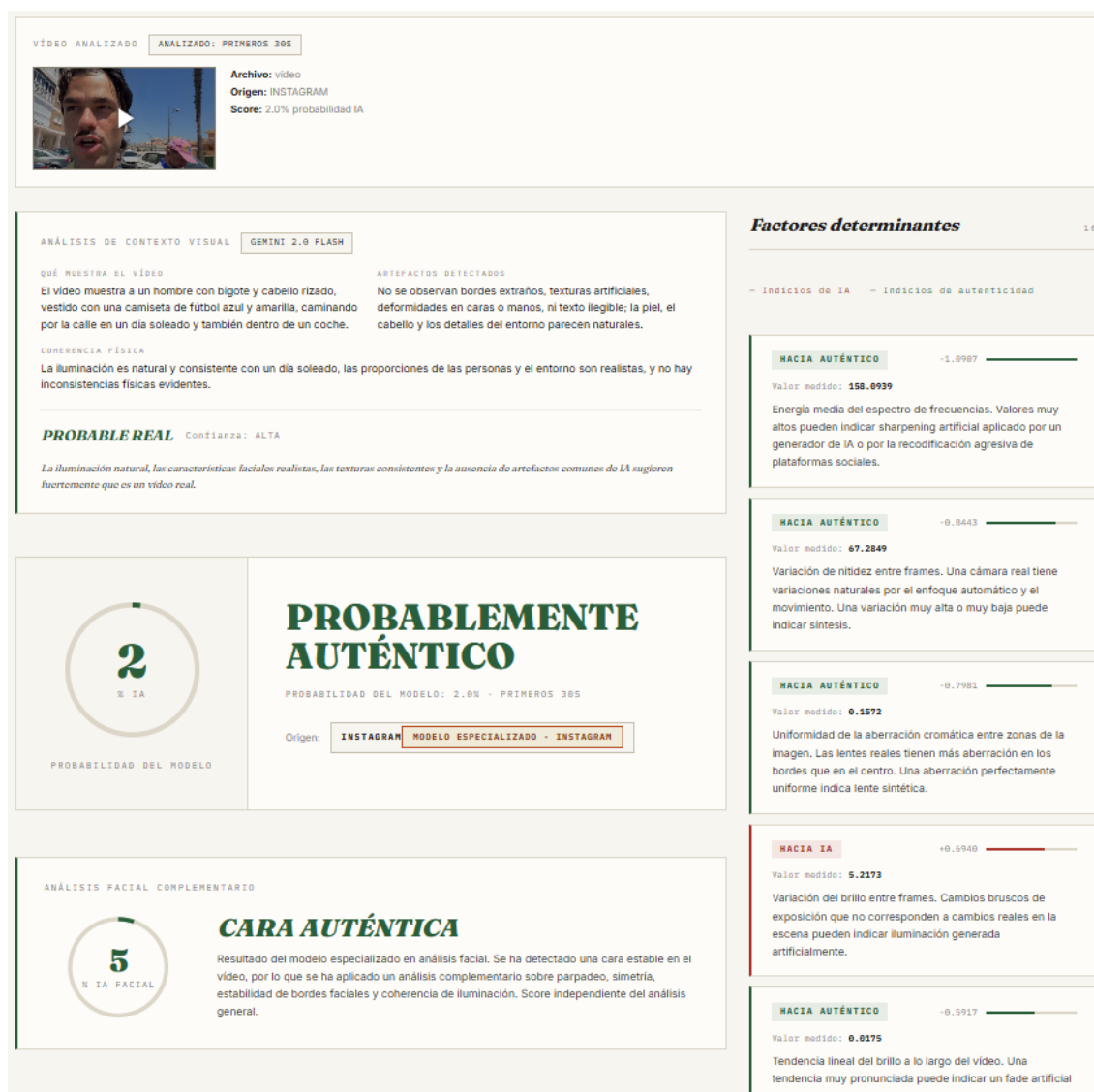


Figura 21. Veredicto completo de vídeo real

De igual manera, el resultado de un vídeo generado por inteligencia artificial sería el siguiente, donde podemos apreciar el vídeo reproduciéndose:

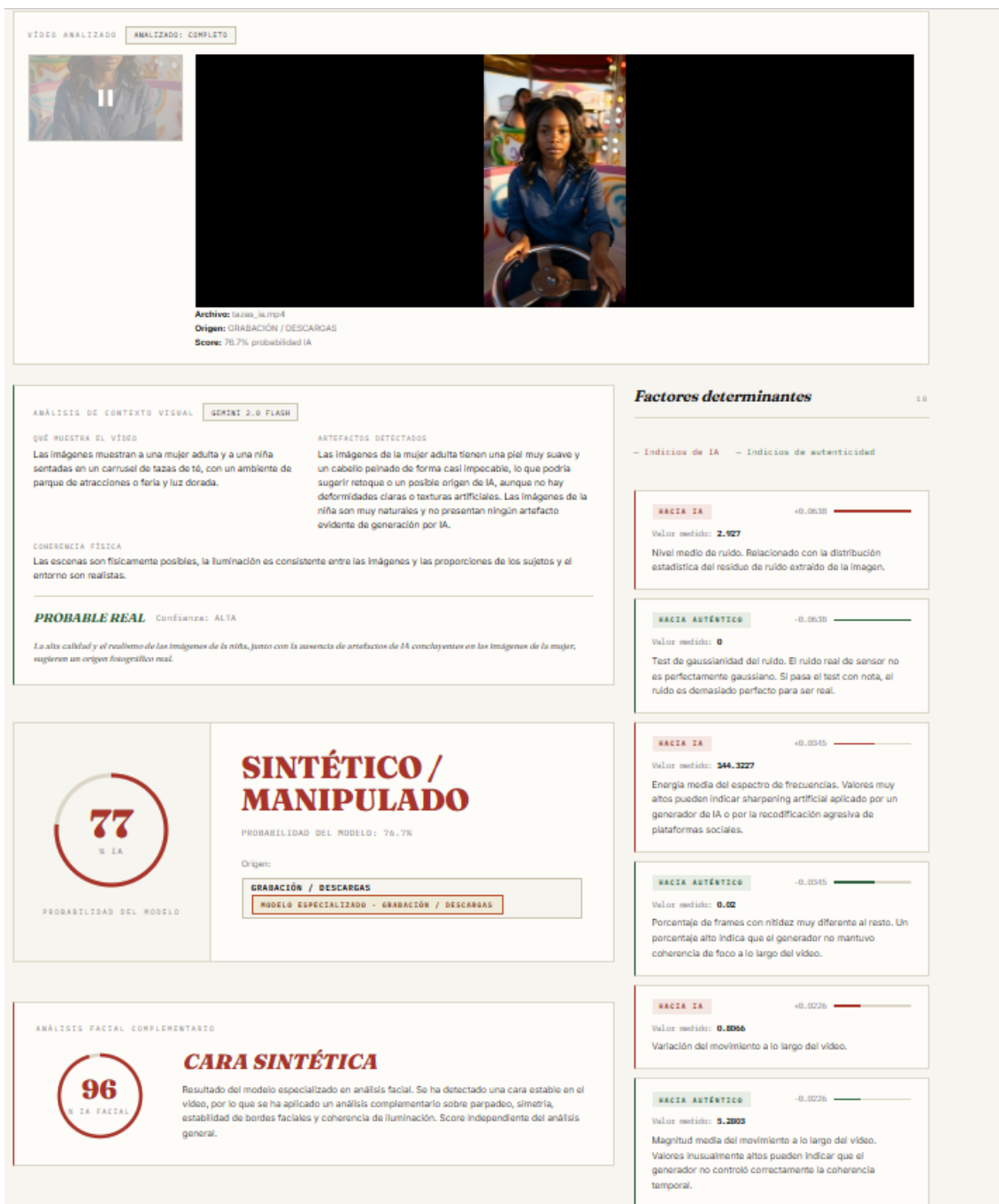


Figura 22. Veredicto completo de vídeo fake

La aplicación también cuenta con una versión adaptada a móviles donde simplemente se apilan las columnas y se adaptan al formato de una pantalla de móvil:



Figura 23. Interfaz de la aplicación en móvil

Capítulo 5. Resultados y validación

En este apartado se presentan los resultados de la validación del sistema y un análisis crítico de los mismos. Esta evaluación se realiza sobre un *holdout* independiente en el que los vídeos son procesados secuencialmente a través del mismo flujo de trabajo que el de la aplicación, de manera que las métricas sean las mismas que reflejarían los videos en el sistema real.

5.1 Composición del *holdout*

El *holdout* tenía un total de 150 vídeos divididos de manera equitativa en la primera etapa de la evaluación, donde 11 dieron fallos de procesamiento y se tuvieron que eliminar para que no alteraran los resultados. Las causas fueron dos puntos concretos de la extracción de datos: los vídeos presentaban contenedores corruptos o incompletos que impedían la extracción completa de fotogramas, o bien tenían metadatos malformados que provocaban errores en la llamada a herramientas externas como FFprobe o Exiftool. Quedó entonces un *holdout* de 139 vídeos divididos en 45-44-50.

5.2 Evolución del protocolo de evaluación

La validación se desarrolló en dos etapas, las cuales sirvieron para ver la evolución del protocolo y comprobar que la metodología que decidí a la hora de trabajar sirve para progresar a base de evaluar y mejorar el sistema de manera constante.

La primera vez, el sistema de modelos por plataforma se evaluó sobre un *holdout* inicial de 150 vídeos con un umbral de decisión neutro de 0,5. Esta evaluación dio un AUC global de 0,764 y me dio un dato clave. Con un umbral neutro, el sistema era más considerablemente más fiable detectando contenido real que detectando generación artificial, siendo los falsos negativos el error predominante. El recall lo podemos calcular a partir de la matriz de confusión de la figura 27, obteniendo como resultado un 56.7% muy mejorable, pero que también se acabaría ajustando.

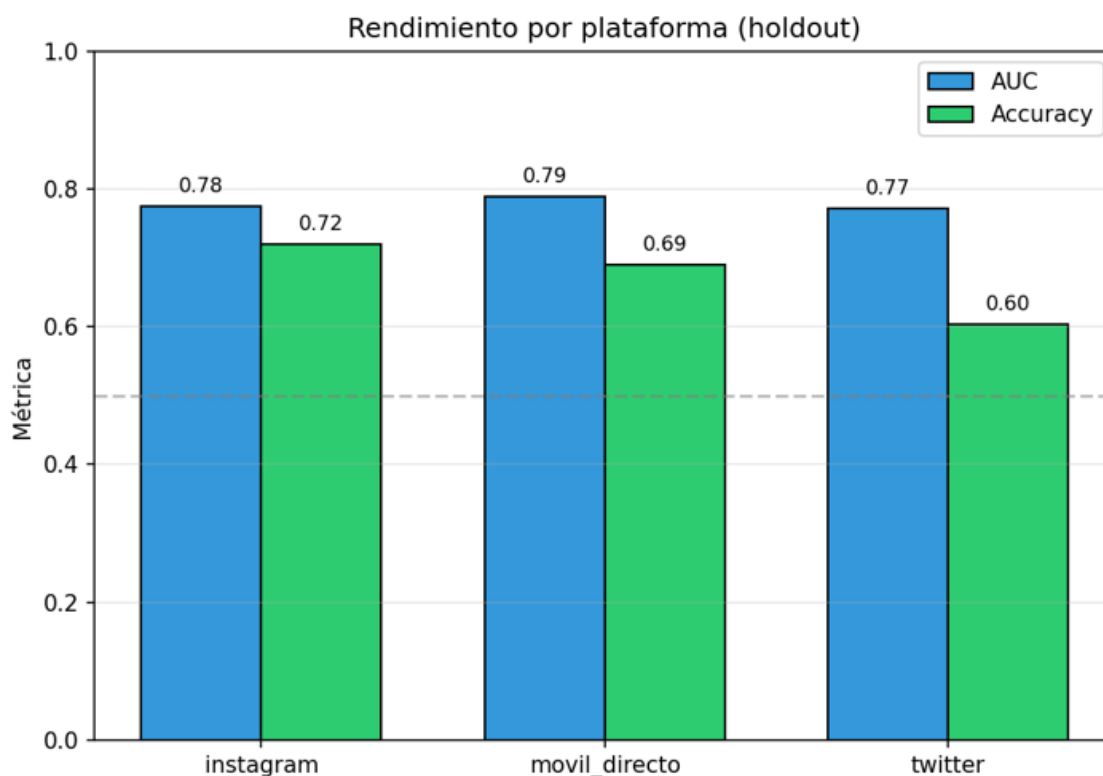


Figura 24. Rendimiento por plataforma (umbral 0.5)

Esta evidencia impulsó el desarrollo de una segunda y definitiva etapa de evaluación. La gráfica de distribución de la figura 26 muestra como una bajada de umbral a valor de 0,4 ayudaría a subir la precisión del sistema a la hora de detectar vídeos artificiales, sacrificando una pequeña parte de falsos positivos, lo cual consideré óptimo frente a tener más falsos negativos.

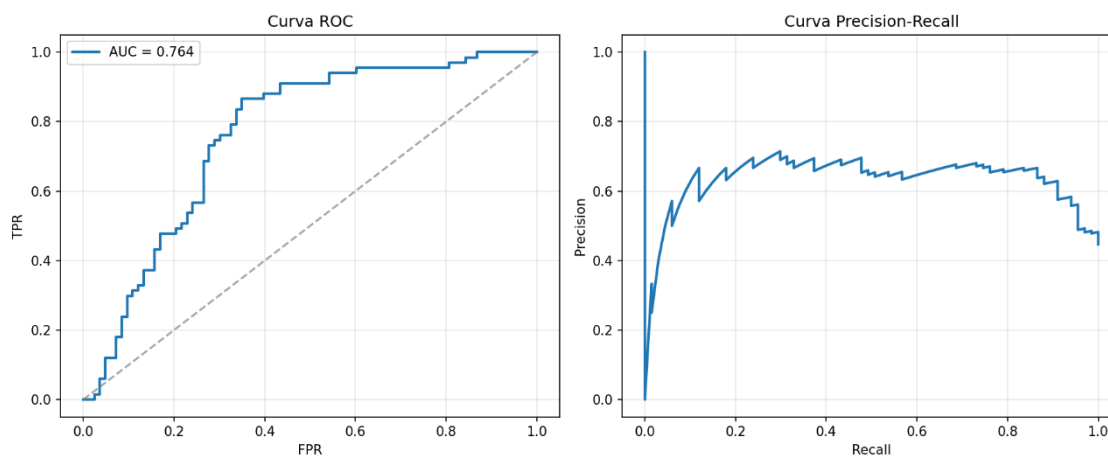


Figura 25. Métricas globales (umbral 0.5)

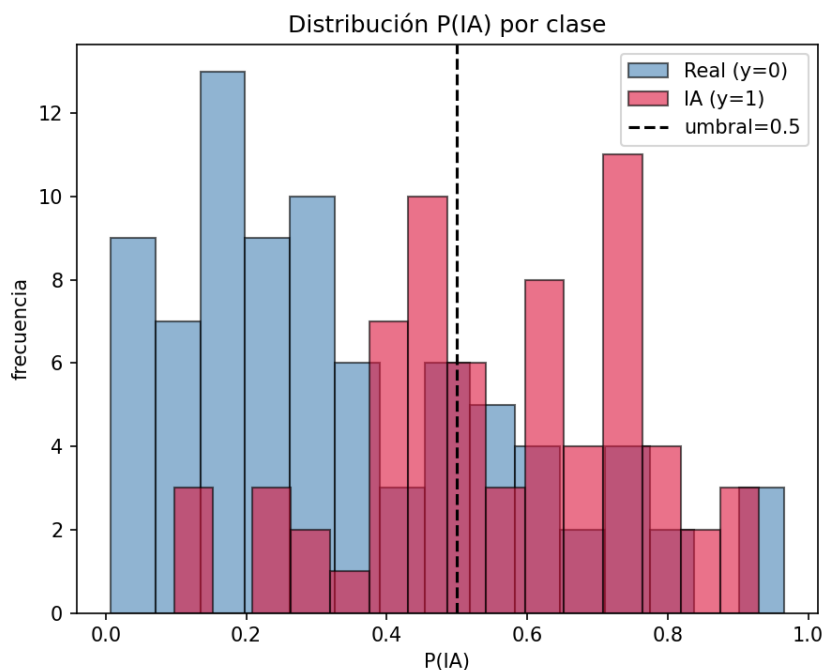


Figura 26. Distribución por plataforma (umbral 0.5)

El reajuste del umbral solo afecta a las métricas dependientes como exactitud, recall y precisión, pero no al AUC, el cual es independiente. La mejora que se observará del AUC se debe a la previamente mencionada limpieza del *holdout*, es decir, a un conjunto homogéneo y equilibrado, sin datos corruptos.

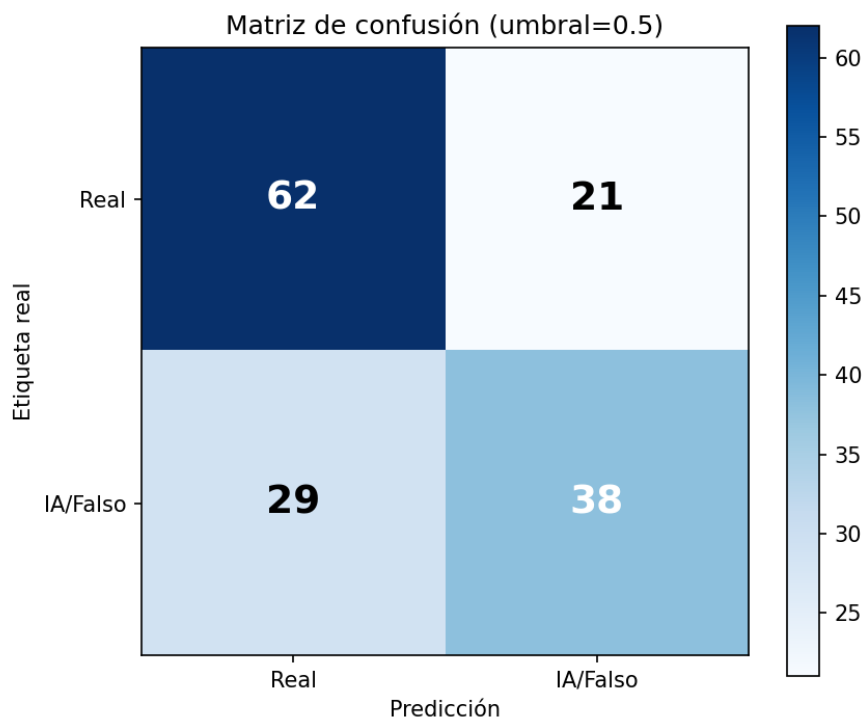


Figura 27. Matriz de confusión global (umbral 0.5)

5.3 Precisión – AUC

La métrica principal de capacidad discriminativa es el área bajo la curva ROC (AUC), porque mide la capacidad del modelo para separar las dos clases independientemente del umbral: representa la probabilidad de que el sistema dé una puntuación mayor a un vídeo sintético aleatorio frente a uno real.

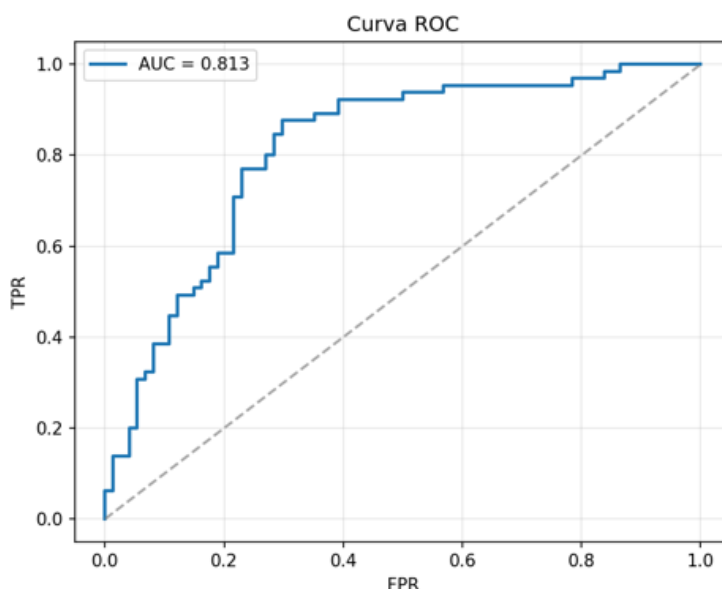


Figura 28. AUC global (umbral 0.4)

En la evaluación definitiva, sobre los 139 vídeos, el sistema alcanza un AUC general de 0.813, claramente por encima de 0.5 que representa el azar, y por encima del AUC de 0.764 de la primera etapa. Además de la media, la importancia reside en el equilibrio entre plataformas, que representa el resultado más importante para esta parte del trabajo. El AUC se mantiene en un intervalo de 0.81-0.87.

Esta regularidad es lo que perseguía el cambio de diseño a modelos especializados. Si hubiera mantenido el modelo unificado como modelo principal de detección, cabría esperar un AUC variante entre orígenes, porque la importancia de cada métrica no es la misma en todas las plataformas debido a las alteraciones que sufren los vídeos en el proceso de subida y descarga. El hecho de conseguir que las tres plataformas rindan de forma comparable da validez al cambio de rumbo: es más efectivo tratar el origen como criterio para elegir un modelo y no como una característica más del clasificador.

Este resultado hay que analizarlo también dentro de su contexto y sus capacidades. Los detectores de extremo a extremo basados en CNN son capaces de alcanzar AUC altísimos que superen 0.95 sobre entornos controlados, pero cuyo rendimiento cae al tener que analizar contenido real que circula por redes. Teniendo en cuenta esto, un AUC de 0.813 obtenido sobre contenido variado extraído de redes sociales es un resultado honesto y robusto.

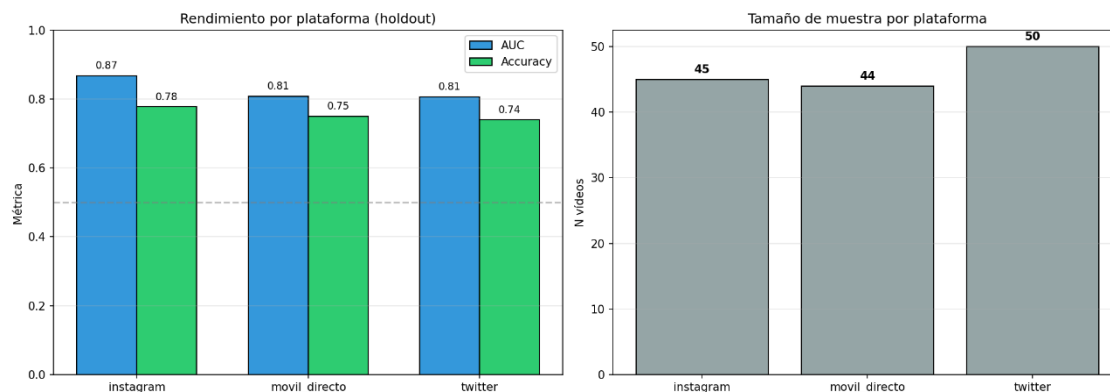


Figura 29. Rendimiento por plataforma (umbral 0.4)

5.4 Diagnóstico de sesgo de dominio (LOPO) y resultados del modelo unificado

El AUC se obtiene a partir de vídeos nuevos de plataformas que el modelo ha visto pero para comprobar que ocurre ante una plataforma completamente desconocida he aplicado la validación ya mencionada Leave-One-Platform-Out.

El contraste de resultados revela una serie de factores a tener en cuenta. Mientras que el mejor modelo alcanza un AUC de 0.866 en validación cruzada estratificada, en LOPO, el AUC desciende a 0.640 en Instagram, 0.608 para grabación/descargas y 0.736 para Twitter, con una media de 0.661. La caída es de aproximadamente 0.205, muy por encima del umbral de 0.10 que fijé como indicador de sesgo de dominio relevante.

La interpretación que hay que hacer con esto es que, dentro del contenido ya visto, el sistema generaliza bien, pero la caída del LOPO me dio que pensar que en lugar de aprender por origen y ser capaz de distinguir, el clasificador se quedaría a medias entre plataformas, sin llegar a ser fiable en ninguna de ellas. Este comportamiento es coherente con los datos sobre importancia de métricas expuestos en el apartado 5.7, comportamiento el cual se explica en parte por el peso que el modelo otorga a las *features* de estructura de codificación, que codifican la huella de recompresión característica de cada plataforma.

Este pequeño estudio muestra una limitación real de las primeras ideas y fue lo que me llevó a apartar el modelo unificado como solución principal y a adoptar el esquema de modelos especializados por plataforma, conservando el unificado como salvavidas cuando no conozcamos el origen del vídeo.

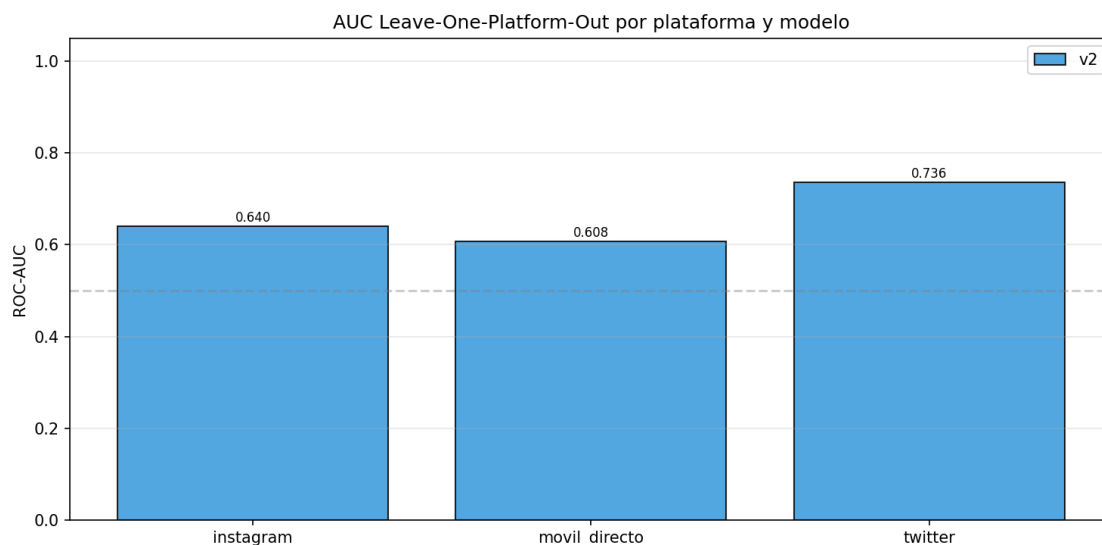


Figura 30. Resultados evaluación LOPO

Además, la evaluación del modelo unificado reafirma lo que nos muestra la evaluación LOPO, con unos resultados bastante inferiores respecto a los obtenidos finalmente:

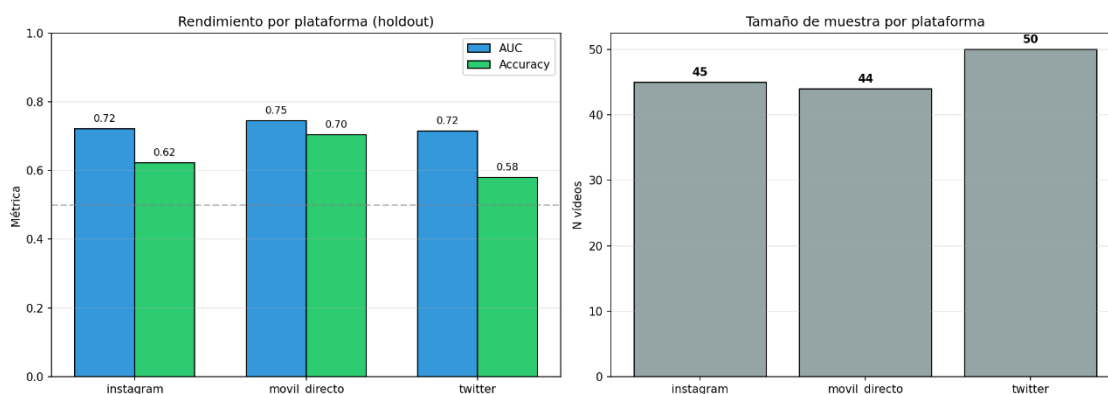


Figura 31. Rendimiento por plataforma modelo unificado (umbral 0.5)

La imagen nos muestra un AUC muy equilibrado entre las 3, coherente si estamos hablando de un modelo unificado, pero con una accuracy bastante baja (con un umbral de 0.5) y un AUC entre 0.10 y 0.12 por debajo del conseguido finalmente por plataforma.

Su matriz de confusión general también es bastante reveladora, acertando mucho en los vídeos reales, pero dejándose fuera muchos vídeos sintéticos, lo cual se vería mejorado por una disminución del umbral, pero aun así muy lejos de ser óptimo.

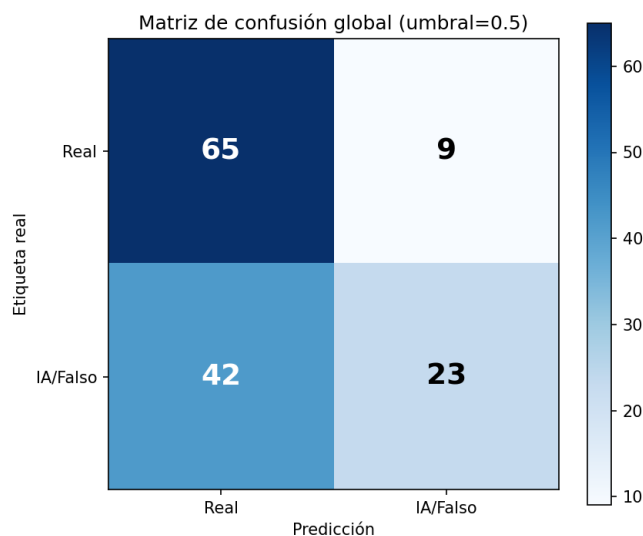


Figura 32. Matriz de confusión modelo unificado

Con el contraste entre unos resultados y otros, se justifica el uso de modelos específicos para que el sistema sea lo más preciso posible.

5.5 Precisión – Recall

Mientras que el AUC mide la capacidad de diferenciar vídeos reales de falsos, el rendimiento real sí que depende del umbral elegido. Como se ha mostrado en el apartado anterior, el umbral definitivo es de 0,4 con el objetivo de elevar la precisión sobre la clase sintética sin sacrificar rendimiento general del sistema.

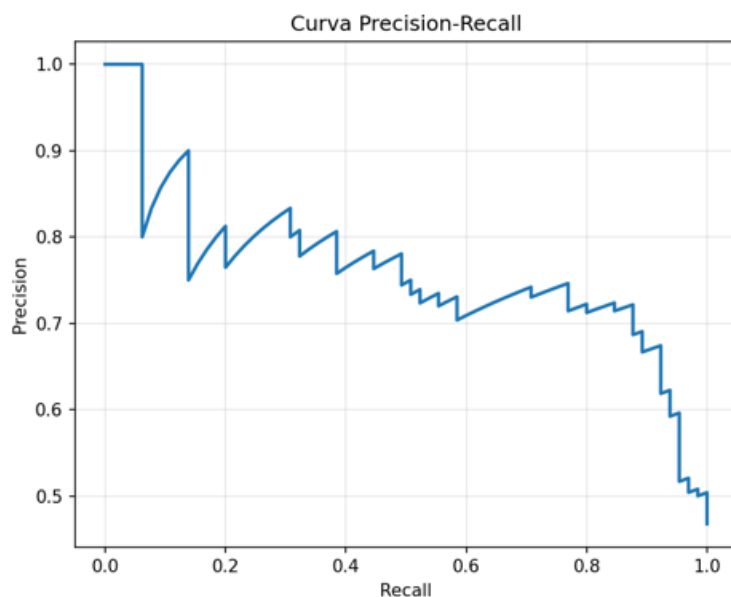


Figura 33. Recall global (umbral 0.4)

El efecto de la calibración es claro al contrastar con las figuras de la primera etapa. Con un umbral neutro, el recall se situaba alrededor del 56,7%; con el ajuste a 0,4 el recall se eleva hasta el 80%,

lo que significa que el sistema detecta alrededor de cuatro de cada cinco vídeos sintéticos respecto a poco más de la mitad anterior. Esta mejora se consigue sin degradar la precisión, que pasa del 64,4% al 71,2% y elevando la exactitud global del 66,7% al 75,5%. La razón por la que precisión y recall mejoran conjuntamente es que se combinan varios factores: el reequilibrio del conjunto y el desplazamiento del umbral.

Los datos exactos los obtuve a partir de los siguientes cálculos basados en la matriz de confusión de la figura X:

Siendo VP (Verdaderos Positivos), VN (Verdaderos Negativos), FP (Falsos Positivos) y FN (Falsos Negativos):

- **Exactitud** = $(VP + VN) / \text{total}$
- **Precisión (IA)** = $VP / (VP + FP)$
- **Exhaustividad / recall (IA)** = $VP / (VP + FN)$
- **F1 (IA)** = $2 \cdot P \cdot R / (P + R)$
- **Especificidad (real)** = $VN / (VN + FP)$

Métrica	Valor	Interpretación
Exactitud global/Accuracy	75.5%	Aciertos sobre el total
Precisión (IA)	71.2%	De los marcados como <i>fake</i> , cuántos lo son
Exhaustividad/recall (IA)	80.0%	De los <i>fakes</i> totales, cuántos se detectan
F1 (IA)	75.4%	Media armónica de precisión y recall
Especificidad (real)	71.6%	De los reales, cuantos se clasifican como reales

La elección de un umbral inferior al neutro se justifica entendiendo lo que significan los errores en contexto del análisis forense: un falso negativo (vídeo falso declarado como auténtico) representa una amenaza con consecuencias más graves en la mayoría de escenarios que un falso positivo.

5.6 Matrices de confusión

La matriz de confusión definitiva es la siguiente:

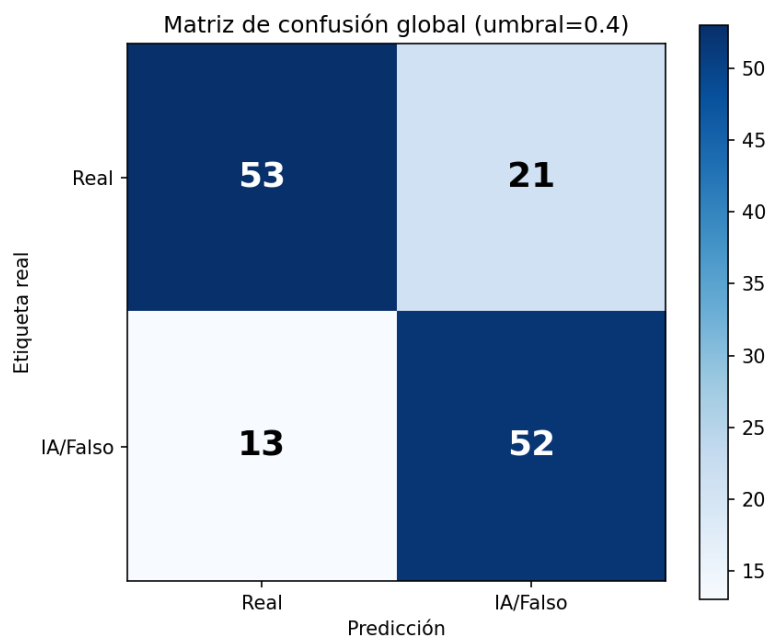


Figura 34. Matriz de confusión global (umbral 0.4)

Esta figura con la distribución de errores vuelve a confirmar los efectos positivos de la calibración del umbral. Los falsos negativos se reducen a 13 respecto a los 29 de la primera fase, por lo que al sistema se le escapan muchos menos vídeos sintéticos. Los falsos positivos se mantienen, prueba de que la mejora de precisión no se ha conseguido como resultado de degradar la fiabilidad sobre el contenido real. La distribución de probabilidades $P(IA)$ por clase que observamos en la figura X muestra el patrón esperado: la clase real gira en torno a valores bajos principalmente mientras que la sintética lo hace sobre valores altos, con un ligero solapamiento alrededor del umbral. Esos vídeos representan los casos de mayor dificultad: vídeos reales fuertemente comprimidos y vídeos *fake* obtenidos de la última generación de generadores.

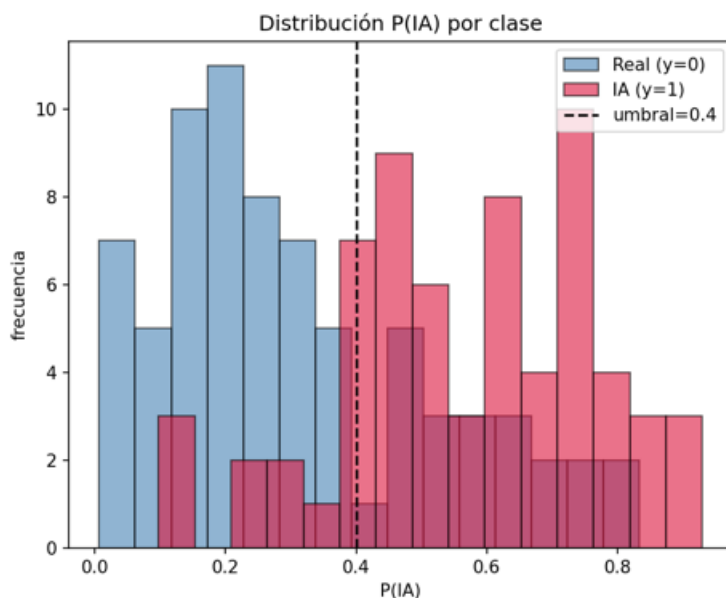


Figura 35. Distribución de probabilidades de IA por clase (umbral 0.4)

Además, también podemos ver las métricas y matrices de confusión por origen, lo que aporta una observación interesante sobre el análisis de los errores. Las matrices muestran que Instagram y Grabaciones/Descargas detectan casi todos los vídeos sintéticos, ya que ambas cuentan con solo 2 falsos negativos cada una, mientras que Twitter acumula 9 de los 13 falsos negativos globales, lo que baja su recall. Es decir, la mayoría de los vídeos *fake* que el sistema pierde proceden de Twitter. Esto encaja con lo que ya sabíamos: la recompresión agresiva de Twitter, mayor que la de Instagram, altera las huellas físicas que busca el modelo. Es coherente también que Twitter presente la mayor precisión (80%) y el menor recall (64%); cuando el sistema marca un vídeo de Twitter como sintético acierta casi siempre, pero se le escapan más.

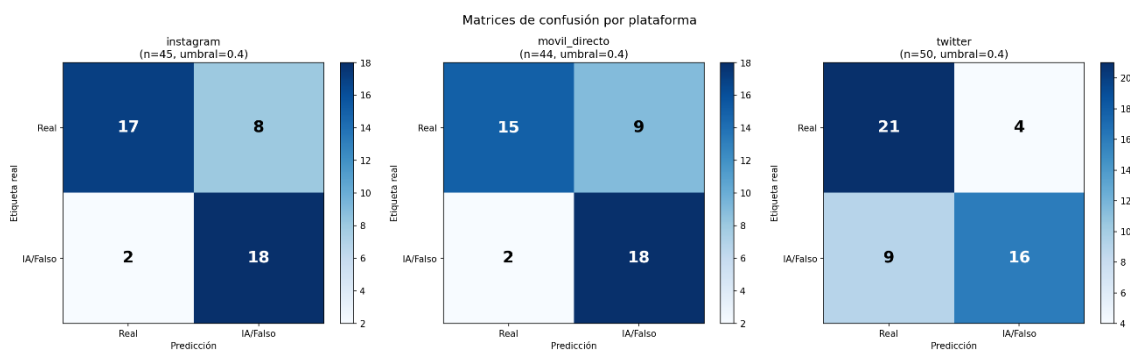


Figura 36. Matrices de confusión por plataforma (umbral 0.4)

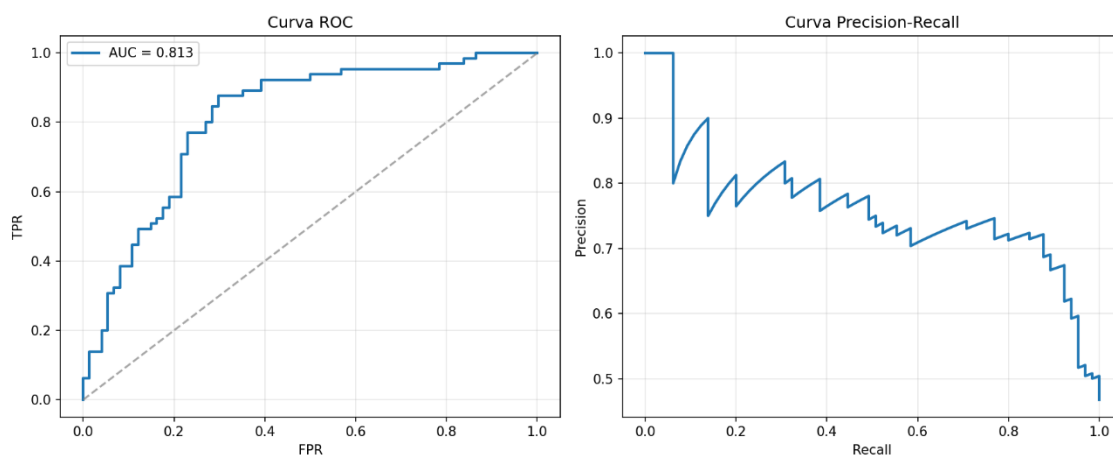


Figura 37. Métricas globales (umbral 0.4)

5.7 Importancia de métricas

El análisis de importancia de las características no solo explica en qué se apoya cada modelo a la hora de asignar puntuaciones, sino que aporta una justificación de la elección del esquema de modelos especializados.

Lo más relevante es que cada plataforma se discrimina mediante *features* distintas, algo con lo que un solo modelo no podría trabajar de manera óptima.

Las etiquetas que se observan a continuación responden a los siguientes significados:

Etiqueta	Significado
nitidez_media	Nitidez media del vídeo
nitidez_std	Desviación típica de la nitidez entre fotogramas

nitidez_outlier_pct	Porcentaje de fotogramas con nitidez atípica
fft_energia_media	Energía media del espectro
fft_ratio_alta_baja	Relación entre altas y bajas frecuencias
fft_ratio_std	Variabilidad temporal de esa relación
flujo_medio	Magnitud media del flujo óptico
flujo_std	Desviación típica del flujo óptico
flujo_salto_maximo	Salto máximo de flujo entre fotogramas consecutivos
flujo_num_glitches	Número de saltos anómalos
brillo_tendencia	Pendiente lineal del brillo a lo largo del vídeo
brillo_std	Variabilidad del brillo medio entre fotogramas
luz_std	Variabilidad de la dirección estimada de iluminación
ca_desfase_medio	Desplazamiento medio rojo-verde (aberración cromática)
ca_desfase_std	Uniformidad de la aberración entre zonas del fotograma
ruido_std	Desviación típica del ruido residual
ruido_asimetria	Asimetría (<i>skewness</i>) del ruido
ruido_curtosis	Curtosis del ruido
corr_rg	Correlación de Pearson entre canales rojo y verde
corr_gb	Correlación de Pearson entre canales verde y azul
ipb_pct_i	Porcentaje de fotogramas I (codificados independientemente)
ipb_pct_p	Porcentaje de fotogramas P (predichos del anterior)
ipb_pct_b	Porcentaje de fotogramas B (predichos en ambas direcciones)
ipb_gop_medio	Tamaño medio del grupo de imágenes (GOP)
ipb_gop_std	Variabilidad de la cadencia del GOP
ipb_ratio_tamano_ip	Ratio entre el tamaño medio de un I-frame y un P-frame
meta_num_alertas	Número de incoherencias detectadas en los metadatos

Tabla 1. Explicación de etiquetas

En el modelo unificado la importancia se reparte de manera equilibrada entre *features* estructurales y físicas, teniendo como más importantes la desviación de la cadencia del GOP, la nitidez media de la imagen a lo largo del vídeo junto con su desviación máxima y la energía espectral.

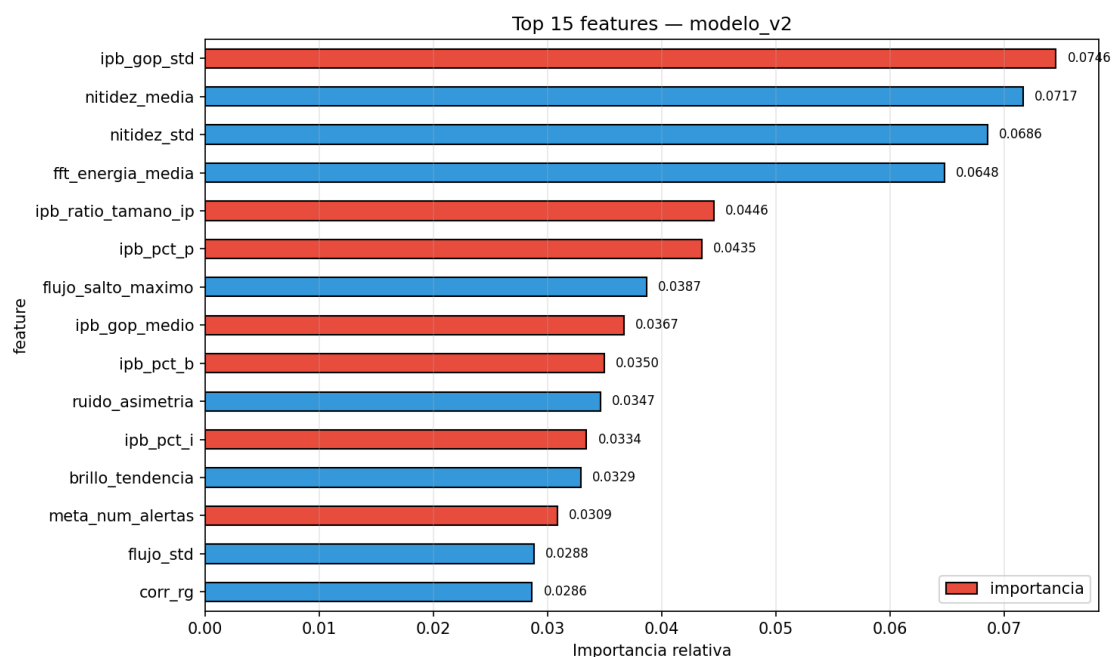


Figura 38. Importancia *features* modelo unificado

En Instagram predominan las evidencias frecuenciales y espaciales, encabezadas por la energía espectral y seguidas por la nitidez media y relación de frecuencias.

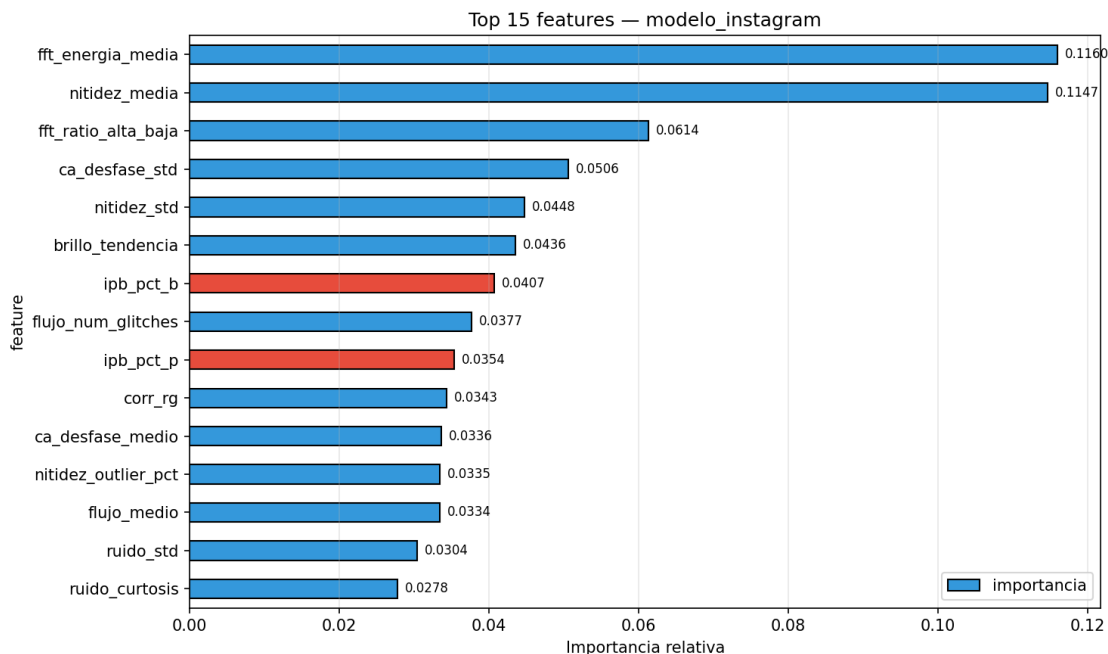


Figura 39. Importancia *features* Instagram

En grabaciones/descargas dominan la estructura de codificación y metadatos, con la desviación del GOP, el salto máximo de flujo óptico y las alertas de metadatos como descriptores principales, resultado lógico en vídeos en su mayoría no reprocesados, cuyo contenedor y metadatos conservan información auténtica que las plataformas destruyen.

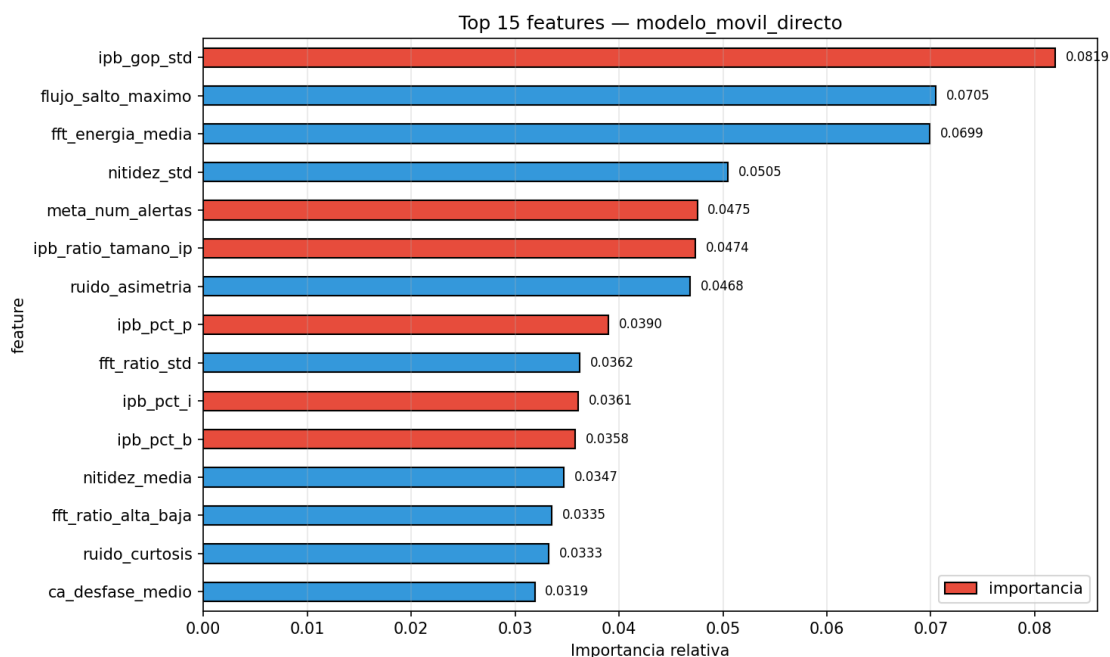


Figura 40. Importancia *features* Grabaciones/Descargas

En Twitter los pesos se reparten entre la nitidez media, la relación de tamaño I/P, el GOP medio y la correlación intercanal RG, que es una huella del proceso de *demosaiicing*.

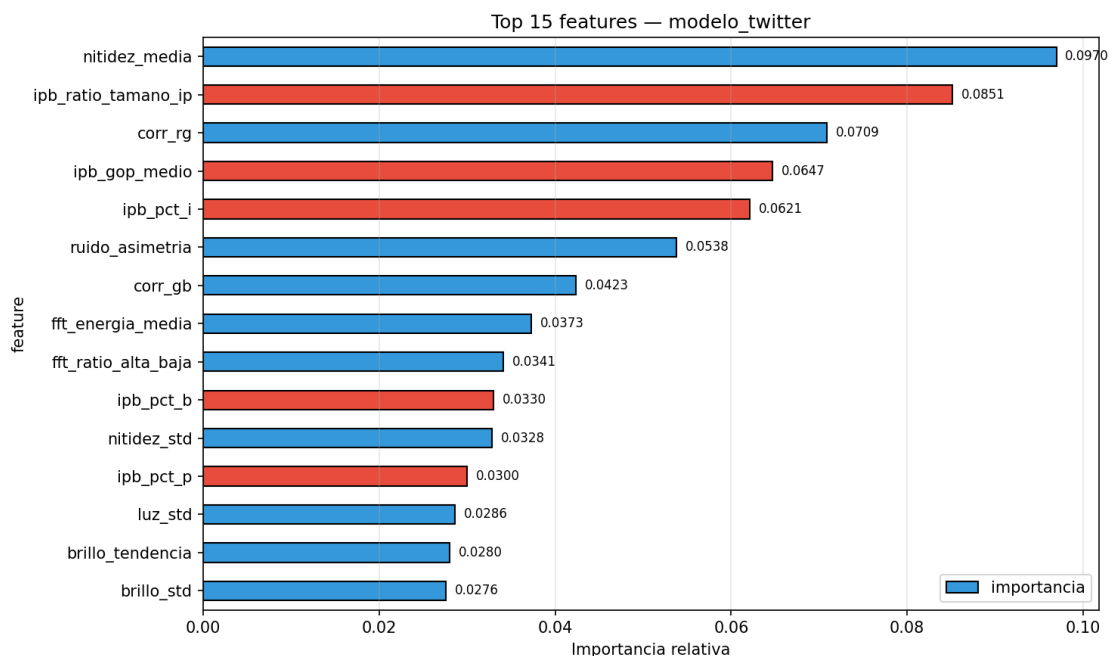


Figura 41. Importancia *features* Twitter

Esta mezcla de *features* tan heterogénea es la justificación del enrutamiento hacia modelos especializados, y también se relaciona directamente con los resultados obtenidos de la evaluación

LOPO. Varios de los parámetros más importantes son de estructura de codificación, donde cada plataforma trabaja de una manera, lo que explica que en plataformas conocidas funcione mejor y se degrade notablemente ante plataformas desconocidas.

5.8 Discusión de resultados

Los resultados de la evaluación final aportan una lectura favorable en general, pero con las limitaciones ya esperadas y comentadas a lo largo del trabajo. Como puntos positivos, el sistema alcanza un AUC de 0,813 con un recall del 80% sobre la clase de IA y una capacidad de reconocimiento equilibrada entre plataformas (0,81-0,87). Este equilibrio respalda la decisión tomada sobre tratar la procedencia como oportunidad de trabajar con varios modelos frente a utilizarla como característica para el clasificador. Por otra parte, el sistema es fácilmente escalable para su aplicación en otras plataformas como Youtube o TikTok, haciendo falta únicamente la construcción manual del *dataset* sobre el que extraer datos y entrenar.

Respecto a las limitaciones, pese a haber conseguido mejorar el sistema inicial, este sigue siendo una herramienta de filtro y asistencia a la verificación y no como una verdad absoluta, lo que significa que recomendable si no se utiliza de manera personal para fines didácticos, siempre será recomendable revisión humana de las detecciones. Hay dos factores que condicionan el alcance de los resultados: la dependencia respecto a los generadores y modelos concretos representados en el entrenamiento (un generador poco visto o un modelo que ha recibido una actualización podrían reducir la fiabilidad del sistema) y la fragilidad de algunas huellas físicas frente a una compresión agresiva, que afecta especialmente a los vídeos reales de las redes sociales.

Estas limitaciones delimitan la contribución del trabajo y orientan el trabajo futuro.

Capítulo 6. Conclusiones y trabajo futuro

6.1 Reflexión personal

Al principio del proyecto, cuando solo era una idea que tenía que descubrir desde que base empezar a construirla, tuve varios cambios de rumbo a medida que descubría las limitaciones del tema tratado, de mis conocimientos y de la investigación de trabajos similares ya desarrollados.

Conocía ya el uso de CNN para esto, pero ni me podía permitir el gasto de tiempo que resulta su entrenamiento para trabajar sobre vídeo, ni me ayudaban a cumplir la parte investigativa y científica del proyecto. Quería descubrir cómo era el vídeo artificial más allá de lo que vemos, buscar maneras de poder aportar pruebas numéricas plausibles más allá de dar un veredicto sin más.

La reciente evolución de los sistemas de generación de vídeo sintético fue lo que me llevó a querer desarrollar este proyecto, pero a su vez han avanzado tanto que han complicado mucho el desarrollo del trabajo. Una vez descubrí que cualquier tipo de *dataset* público estaba desactualizado, tuve que adaptar la planificación del trabajo para poder integrar la creación manual del *dataset* que iba a utilizar, ya que esto me iba a quitar bastante tiempo por la necesidad de revisar manualmente los vídeos, descartando aquellos que generaran muchas dudas o que fueran excesivamente evidentes, y procurando que su contenido fuera variado, ya que los vídeos que te puedas encontrar a través del algoritmo eran muy similares ya que estos funcionan por modas.

Sobre la extracción de datos, fue creciendo poco a poco el número de los mismo a base de probar, repetir y ampliar, descubriendo que a través de un mismo valor se podían extraer varias métricas aprovechables, en lugar de tener que buscar 30 métricas basadas en 30 tipos de datos espaciotemporales diferentes.

Por último, este proyecto me ha confirmado lo que ya pensaba antes, y es que la existencia de un proyecto así es un síntoma bastante malo. La inteligencia artificial es una herramienta que está cambiando el mundo a pasos agigantados y que es muy útil en muchos ámbitos desde científicos y médicos a tecnológicos, pasando por su uso en el día a día. Pero la rama de la creación audiovisual me parece la excepción y donde hay que marcar la línea.

Sobre la parte menos nociva, su uso con fines artísticos y de entretenimiento ha provocado que mucha gente muestre el mismo aprecio por una imagen generada que por un dibujo o cuadro real, donde una persona ha puesto esfuerzo y emoción por poder plasmar y transmitir su idea y sentimientos, y lo mismo aplicable a vídeo y audio a través de animaciones o música, lo que lleva a devaluar el valor del arte real que lleva acompañando nuestra historia desde siglos y siglos.

Si hablamos de un aspecto más dañino, entramos en un terreno bastante más grave. He mencionado en este documento que se ha extendido su uso de *fake news* a estafas de millones de dólares y a la sexualización y exposición no consentida (además de falsa) del físico de las personas, especialmente de mujeres y niños. He visto muchos vídeos a lo largo de estos meses, lo que me ha llevado a descubrir estafas de menor escala a través de cursos donde te prometen ganar miles de euros mediante la generación de vídeos artificiales. También canales privados de pago donde hombres se crean avatares digitales femeninos para vender su contenido. Y los *fakes* sobre desnudos son la peor parte, ya que estos pueden causar un daño permanente a la imagen y la salud mental de las personas afectadas, y que además está totalmente penado en España.

Si para convivir con esta sección de la inteligencia artificial hay que aceptar estas consecuencias y consumir tantos recursos, bajo mi punto de vista, nos estaremos equivocando como sociedad.

6.2 Cumplimiento de objetivos

El objetivo principal, que era construir un detector que combinara análisis de datos forenses interpretables con clasificadores binarios, tratando la generalización entre plataformas y que aportara explicaciones sobre los resultados, todo bajo una aplicación web funcional; lo considero alcanzado. El resultado es un sistema que procesa un vídeo de extremo a extremo, da un veredicto justificado, y cuyo funcionamiento global ha sido evaluado sobre un *holdout* independiente del entrenamiento basado en el propio flujo del funcionamiento de la aplicación, alcanzando un AUC global de 0.813 y una exhaustividad del 80% sobre la clase sintética.

Sobre los objetivos técnicos, estoy satisfecho con su implementación. El proyecto implementa la canalización de extracción de métricas forenses mixtas (espaciales, temporales) para el entrenamiento comparado de cuatro tipos de clasificadores, seleccionando los métodos basados en árboles por su rendimiento.

El riesgo de sesgo de dominio se ha evaluado mediante LOPO, con unos resultados concretos que validan el pensamiento que hay detrás y que, viendo los resultados posteriores, lo considero una medición valorable y rigurosa. El desarrollo de diferentes modelos terminó de validar este pensamiento y su aplicación. El conjunto termina con la integración de explicaciones mediante SHAP y un análisis de contexto mediante un modelo multimodal, todo presentado a través de una web funcional y limpia que cumple su función, cerrando correctamente el conjunto de objetivos definidos al inicio del proyecto.

Los objetivos secundarios también los considero cumplidos en buena medida. El sistema es fácilmente escalable en caso de querer integrarse más redes o plataformas como Youtube o Tiktok. También se ha podido probar el análisis facial a través de señales fisiológicas, con un desarrollo menos profundo pero que sirve de base útil para entender que sería posible y complementario como ampliación del flujo de trabajo principal.

6.3 Aportaciones e impacto

Más allá de cumplir con mis objetivos, considero que he conseguido desarrollar una herramienta que, si alguna vez se hiciera pública y accesible a todo el mundo, podría ser muy utilizada y tendría un impacto real en cómo la gente interactuaría con las redes sociales. Desde el punto de vista de los jóvenes que convivimos con la tecnología y de los profesionales que trabajan con ella, es complicado comparar como vemos y entendemos nosotros los vídeos que te puedes cruzar por internet, pero para una persona cualquiera que simplemente sea consumidora de redes sociales sin un conocimiento más allá, es muy fácil creerte todo lo que hay por internet. Esto lo he podido comprobar a través de mis padres, que indirectamente fueron los que dieron la idea y provocaron esa chispa que ha llevado al desarrollo de este TFG.

Todo lo que se expone en los resultados del análisis de un vídeo puede llevar a que, poco a poco, hasta los más desconocedores sean capaces en cierta medida de discernir si lo que están viendo es real y posible o no, ya que esto lo pueden aprender a través de las explicaciones de contexto o los resultados faciales, no necesariamente tiene que entender el resultado de las *features* más técnicas. Como vamos a convivir con esta tecnología muchos años, un remedio para evitar confusiones o engaños es darle a la gente una vía para aprender y comprender.

6.4 Limitaciones del sistema

Desde el principio del trabajo se asumen unas limitaciones consecuencia de la naturaleza de este.

El sistema actualmente se encuentra en el rango de una herramienta de cribado y asistencia a la verificación, no un detector concluyente con un resultado de altísima fiabilidad: una precisión del 71% implica que una proporción de las alertas va a requerir revisión manual. Además, está construido alrededor de un *dataset* de tamaño moderado, lo cual limita el rendimiento actual del sistema.

Su rendimiento depende de los generadores que hay actualmente, siendo incierta su aplicación en vídeos de generadores de dentro de un año, lo que podría degradar el rendimiento a futuro si no se va actualizando y reentrenando.

Y, por último, la velocidad de procesamiento del sistema. Si sospechas que un vídeo es falso, con un fragmento sería más que suficiente para determinarlo, pero en caso de ejecutar el programa con un vídeo de larga duración, el sistema tardaría mucho tiempo en devolver un resultado, exponiéndote a que falle por el camino.

6.5 Trabajo futuro

El estado actual de la aplicación hace que fueran evidentes algunas mejoras posibles que requirieran un desarrollo en profundidad

6.5.1 Extensión de navegador

La posibilidad de publicar la aplicación como una extensión de navegador proporcionaría al usuario la capacidad de llamar al flujo de trabajo principal mientras está viendo las redes, de manera que, por ejemplo, si ve un vídeo que querría introducir en la web, a través de la extensión podría simplemente leer la URL del navegador y devolver un resultado en formato reducido integrado en la propia interfaz.

6.5.2 Robustez contra compresiones agresivas y ataques

El trabajo podría incluir una evaluación de robustez sistemática: medir la caída de rendimiento tras varias compresiones de archivo, reescalados o filtrados que simulen su publicación múltiples veces por diferentes plataformas. Los resultados servirían para reforzar el sistema aumentando la base de datos de entrenamiento, para que el modelo aprenda a reconocer estas manipulaciones.

6.5.3 Análisis facial

Ya se ha mencionada que el apartado respecto al análisis de caras está en fase de desarrollo y como complemento, pero no ha recibido el mismo nivel de validación ni integración que el núcleo del sistema. Llevarlo de fase preliminar a un componente definitivo le daría mucho valor y fiabilidad al trabajo, siendo además la suplantación de personas una de las fuentes de contenido más sensibles.

Primero, habría que darle una validación similar a la que se ha hecho con el sistema principal con un *holdout* de vídeos específico, documentando AUC, precisión, recall, F1 y matriz de confusión.

Y después, la ampliación de los descriptores fisiológicos, como por ejemplo la dirección de la mirada, la consistencia de la pose de la cabeza o la coherencia de las expresiones.

6.5.4 Incorporación de análisis de audio

El sistema actualmente solo tiene en cuenta el vídeo. Pero la mayoría de las veces, el vídeo suele venir acompañado de un audio, y una cantidad importante de generadores ya llevan incluida la capacidad de generar audio de calidad y sincronizado, pero es un campo que podría dar para un proyecto independiente en sí mismo.

Como propuesta, podría añadir análisis espectral de la voz y detección de artefactos de síntesis vocal, y de forma complementaria al análisis facial, la sincronía labial entre el movimiento de la boca y el audio, pudiendo ser este un indicio bastante sólido de manipulación.

6.5.5 Fusión del veredicto

El sistema presenta tres fuentes de análisis donde cada una da un veredicto independiente, pudiendo fusionar a través de un clasificador entrenado que aprenda la mejor ponderación de cada evidencia en función de su fiabilidad contextual, por ejemplo, dando más peso al análisis facial en un vídeo de un primer plano nítido.

Capítulo 7. Bibliografía

- [1] B. Allyn, «*Deepfake* video of Zelenskyy could be 'tip of the iceberg' in info war, experts warn,» NPR, 2022. [En línea]. Available: <https://www.npr.org/2022/03/16/1087062648/deepfake-video-zelenskyy-experts-war-manipulation-ukraine-russia>.
- [2] «Arup revealed as victim of \$25 million *deepfake* scam involving Hong Kong employee,» CNN Business, 2024. [En línea]. Available: <https://www.cnn.com/2024/05/16/tech/arup-deepfake-scam-loss-hong-kong-intl-hnk>.
- [3] «Rosalía y JC Reyes: la polémica que vuelve a dejar claro que el cuerpo de la mujer 'no es una mercancía',» Infobae, 2023. [En línea]. Available: <https://www.infobae.com/espana/2023/05/25/rosalia-y-jc-reyes-la-polemica-que-vuelve-a-dejar-claro-que-el-cuerpo-de-la-mujer-no-es-una-mercancia/>.
- [4] N. C. Köbis, B. Doležalová e I. Soraperra, «Fooled twice: people cannot detect *deepfakes* but think they can,» iScience, 2021. [En línea]. Available: <https://doi.org/10.1016/j.isci.2021.103364>.
- [5] Hive AI, «AI-generated & *deepfake* content detection,» Hive AI, [En línea]. Available: <https://thehive.ai/apis/ai-generated-content-classification>.
- [6] Reality Defender, «Platform / Technology,» Reality Defender, [En línea]. Available: <https://www.realitydefender.com/platform/technology>.
- [7] Sensity AI, «*Deepfake* detection for video, image & audio,» Sensity AI, [En línea]. Available: <https://sensity.ai/deepfake-detection-for-video-image-audio/>.
- [8] Truepic, «Content provenance,» Truepic, [En línea]. Available: <https://www.truepic.com/>.
- [9] H. Ajder, G. Patrini, F. Cavalli y L. Cullen, «The State of *Deepfakes*: Landscape, Threats, and Impact,» Deeptrace, 2019.
- [10] I. J. Goodfellow et al., «Generative adversarial networks,» arXiv, 2014. [En línea]. Available: <https://arxiv.org/abs/1406.2661>.
- [11] T. Karras, S. Laine y T. Aila, «A style-based generator architecture for generative adversarial networks,» arXiv, 2019. [En línea]. Available: <https://arxiv.org/abs/1812.04948>.
- [12] T. Karras et al., «Analyzing and improving the image quality of StyleGAN,» arXiv, 2020. [En línea]. Available: <https://arxiv.org/abs/1912.04958>.
- [13] A. Odena, V. Dumoulin y C. Olah, «Deconvolution and checkerboard artifacts,» Distill, 2016. [En línea]. Available: <https://distill.pub/2016/deconv-checkerboard/>.
- [14] J. Ho, A. Jain y P. Abbeel, «Denoising diffusion probabilistic models,» arXiv, 2020. [En línea]. Available: <https://arxiv.org/abs/2006.11239>.
- [15] R. Rombach et al., «High-resolution image synthesis with latent diffusion models,» arXiv, 2022. [En línea]. Available: <https://arxiv.org/abs/2112.10752>.

- [16] T. Brooks et al., «Video generation models as world simulators,» OpenAI, 2024. [En línea]. Available: <https://openai.com/index/video-generation-models-as-world-simulators/>.
- [17] W. Peebles y S. Xie, «Scalable diffusion models with transformers,» arXiv, 2023. [En línea]. Available: <https://arxiv.org/abs/2212.09748>.
- [18] D. J. Field, «Relations between the statistics of natural images and the response properties of cortical cells,» Journal of the Optical Society of America A, 1987. [En línea]. Available: <https://doi.org/10.1364/JOSAA.4.002379>.
- [19] S.-Y. Wang et al., «CNN-generated images are surprisingly easy to spot... for now,» IEEE/CVF CVPR, 2020. [En línea]. Available: <https://arxiv.org/abs/1912.11035>.
- [20] G. Farneäck, «Two-frame motion estimation based on polynomial expansion,» Image Analysis (SCIA 2003), Springer, 2003. [En línea]. Available: https://doi.org/10.1007/3-540-45103-X_50.
- [21] J. Lukáš, J. Fridrich y M. Goljan, «Digital camera identification from sensor pattern noise,» IEEE Transactions on Information Forensics and Security, 2006. [En línea]. Available: <https://doi.org/10.1109/TIFS.2006.873602>.
- [22] S. S. Shapiro y M. B. Wilk, «An analysis of variance test for normality (complete samples),» Biometrika, 1965. [En línea]. Available: <https://doi.org/10.1093/biomet/52.3-4.591>.
- [23] M. K. Johnson y H. Farid, «Exposing digital forgeries through chromatic aberration,» ACM Workshop on Multimedia and Security, 2006. [En línea]. Available: <https://doi.org/10.1145/1161366.1161376>.
- [24] Y. Li, M.-C. Chang y S. Lyu, «In Ictu Oculi: exposing AI created *fake* videos by detecting eye blinking,» IEEE WIFS, 2018. [En línea]. Available: <https://arxiv.org/abs/1806.02877>.
- [25] F. Pedregosa et al., «Scikit-learn: machine learning in Python,» Journal of Machine Learning Research, 2011. [En línea]. Available: <https://jmlr.org/papers/v12/pedregosa11a.html>.
- [26] L. Breiman, «Random forests,» Machine Learning, 2001. [En línea]. Available: <https://doi.org/10.1023/A:1010933404324>.
- [27] J. H. Friedman, «Greedy function approximation: a gradient boosting machine,» The Annals of Statistics, 2001. [En línea]. Available: <https://doi.org/10.1214/aos/1013203451>.
- [28] T. Chen y C. Guestrin, «XGBoost: a scalable tree boosting system,» ACM SIGKDD, 2016. [En línea]. Available: <https://doi.org/10.1145/2939672.2939785>.
- [29] L. S. Shapley, «A value for n-person games,» en Contributions to the Theory of Games II, Princeton University Press, 1953.
- [30] S. M. Lundberg y S.-I. Lee, «A unified approach to interpreting model predictions,» NeurIPS, 2017. [En línea]. Available: <https://arxiv.org/abs/1705.07874>.
- [31] S. M. Lundberg et al., «From local explanations to global understanding with explainable AI for trees,» Nature Machine Intelligence, 2020. [En línea]. Available: <https://doi.org/10.1038/s42256-019-0138-9>.

- [32] Python Software Foundation, «Python Language Reference,» Python Software Foundation, [En línea]. Available: <https://docs.python.org/3/>.
- [33] C. R. Harris et al., «Array programming with NumPy,» Nature, 2020. [En línea]. Available: <https://www.w3schools.com/python/numpy/default.asp>.
- [34] P. Virtanen et al., «SciPy 1.0: fundamental algorithms for scientific computing in Python,» Nature Methods, 2020. [En línea]. Available: <https://docs.scipy.org/doc/scipy/index.html>.
- [35] W. McKinney, «Data structures for statistical computing in Python,» Python in Science Conference, 2010. [En línea]. Available: <https://pandas.pydata.org/docs/>.
- [36] J. D. Hunter, «Matplotlib: a 2D graphics environment,» Computing in Science & Engineering, 2007. [En línea]. Available: <https://matplotlib.org/stable/users/index.html>.
- [37] G. Bradski, «The OpenCV Library,» Dr. Dobb's Journal of Software Tools, 2000. [En línea]. Available: <https://opencv.org/>.
- [38] FFmpeg Project, «FFmpeg Documentation,» FFmpeg, [En línea]. Available: <https://ffmpeg.org/documentation.html>.
- [39] P. Harvey, «ExifTool by Phil Harvey,» [En línea]. Available: <https://exiftool.org/>.
- [40] C. Lugaresi et al., «MediaPipe: a framework for building perception pipelines,» arXiv, 2019. [En línea]. Available: <https://developers.google.com/edge/mediapipe/solutions/guide>.
- [41] Google, «Gemini 2.5 Flash,» Google AI for Developers, [En línea]. Available: <https://ai.google.dev/gemini-api/docs/models/gemini-2.5-flash>.
- [42] M. Grinberg, «Flask Documentation,» Pallets Projects, [En línea]. Available: <https://flask.palletsprojects.com/>.
- [43] M. W. Docs, «HTML: HyperText Markup Language,» s.f. [En línea]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTML>.
- [44] M. W. Docs, «CSS: Cascading Style Sheets,» s.f. [En línea]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTML>.
- [45] M. W. Docs, «JavaScript,» s.f. [En línea]. Available: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>.
- [46] M. W. Docs, «Using server-sent events,» s.f. [En línea]. Available: https://developer.mozilla.org/es/docs/Web/API/Server-sent_events/Using_server-sent_events.
- [47] yt-dlp, «yt-dlp: a feature-rich command-line audio/video downloader,» GitHub, [En línea]. Available: <https://github.com/yt-dlp/yt-dlp>.

Capítulo 8. Anexos

8.1 Construcción del *dataset* – *extraer_datos.py*

8.1.1 Detección de la plataforma de origen

```
def leer_origen_ydlp(ruta_video):
    json_p = os.path.splitext(ruta_video)[0] + '.info.json'
    if not os.path.exists(json_p):
        return 'movil_directo'
    try:
        extractor = json.load(open(json_p, encoding='utf-8')).get('extractor', '').lower()
    except Exception:
        return 'desconocido'
    for clave in ('instagram', 'twitter'):
        if clave in extractor:
            return clave
    return 'twitter' if 'x.com' in extractor else 'desconocido'
```

Figura 42. Leer origen del vídeo

8.1.2 Extracción de fotogramas y lectura en memoria

```
def extraer_frames(ruta_video, dir_tmp, fps):
    os.makedirs(dir_tmp, exist_ok=True)
    subprocess.run(
        ['ffmpeg', '-y', '-i', ruta_video, '-vf', f'fps={fps}',
         os.path.join(dir_tmp, 'out%d.png')],
        capture_output=True
    )
    return len(glob.glob(os.path.join(dir_tmp, 'out*.png')))

def cargar_frames(dir_tmp):
    """Lee TODOS los frames a memoria una sola vez."""
    frames, i = [], 1
    while True:
        p = os.path.join(dir_tmp, f'out{i}.png')
        if not os.path.exists(p): # evita el warning de OpenCV y el delay
            break
        img = cv2.imread(p, cv2.IMREAD_GRAYSCALE)
        if img is None:
            break
        frames.append(img)
        i += 1
    color = cv2.imread(os.path.join(dir_tmp, 'out1.png')) if frames else None
    return frames, color
```

Figura 43. Extracción y lectura de *frames*

Las imágenes se guardan en escala de grises para tener un procesamiento más ligero en la memoria, puesto que la mayoría de las medidas solo necesitan luminancia, solo la aberración cromática y la correlación entre canales RGB necesitan color.

8.1.3 Procesamiento de los vídeos

```
def procesar_video(args):
    ruta_video, etiqueta, fps = args
    dir_tmp = f"{DIR_TMP}_{os.getpid()}"

    try:
        if extraer_frames(ruta_video, dir_tmp, fps) == 0:
            return None

        frames, color = cargar_frames(dir_tmp)  # ◀ una sola lectura
        if not frames:
            return None

        nit_med, nit_std, nit_out = calc_nitidez(frames)
        fft_e, fft_r, fft_rs = calc_fft(frames)
        fl_med, fl_std, fl_salto, fl_gl = calc_flujo_optico(frames)
        luz_std, br_std, br_tend = calc_fotometria(frames)
        ca_med, ca_std = calc_aberracion_cromatica(color)
        dct_r = calc_dct(frames)
        r_p, r_std, r_asim, r_curt = calc_ruido(frames)
        rg2, rb2, gb2 = calc_correlacion_rgb(color)
        pi, pp, pb, gm, gs, ir, ial = calc_ipb(ruta_video)
        mal = calc_metadatos(ruta_video)
        origen = leer_origen_ydlp(ruta_video)

        return {
            'video': ruta_video, 'etiqueta': etiqueta, 'origen': origen,
            'nitidez_media': nit_med, 'nitidez_std': nit_std, 'nitidez_outlier_pct': nit_out,
            'fft_energia_media': fft_e, 'fft_ratio_alta_baja': fft_r, 'fft_ratio_std': fft_rs,
            'flujo_medio': fl_med, 'flujo_std': fl_std,
            'flujo_salto_maximo': fl_salto, 'flujo_num_glitches': fl_gl,
            'luz_std': luz_std, 'brillo_std': br_std, 'brillo_tendencia': br_tend,
            'ca_desfase_medio': ca_med, 'ca_desfase_std': ca_std,
            'dct_ratio': dct_r,
            'ruido_shapiro_p': r_p, 'ruido_std': r_std,
            'ruido_asimetria': r_asim, 'ruido_curtosis': r_curt,
            'corr_rg': rg2, 'corr_rb': rb2, 'corr_gb': gb2,
            'ipb_pct_i': pi, 'ipb_pct_p': pp, 'ipb_pct_b': pb,
            'ipb_gop_medio': gm, 'ipb_gop_std': gs,
            'ipb_ratio_tamano_ip': ir, 'ipb_num_alertas': ial,
            'meta_num_alertas': mal,
            **codificar_origen(origen),
        }

    except Exception as e:
        print(f" [!] Error en {ruta_video}: {e}")
        traceback.print_exc()
        return None

    finally:
        limpiar_frames(dir_tmp)
        try:
            os.rmdir(dir_tmp)
        except Exception:
```

Figura 44. Función para procesar vídeos

8.2 Entrenamiento del modelo general – entrenar_ia.py

8.2.1 LimpiadorFeatures

```
class LimpiadorFeatures(BaseEstimator, TransformerMixin):

    def __init__(self, sigma_clip=3.0):
        self.sigma_clip = sigma_clip

    def fit(self, X, y=None):
        X = np.asarray(X, dtype=float)
        self.medians_ = np.nanmedian(X, axis=0)
        # mean/std tras imputar para que sean estables ante NaN
        X_imp = np.where(np.isnan(X), self.medians_, X)
        self.means_ = X_imp.mean(axis=0)
        self.stds_ = X_imp.std(axis=0)
        return self

    def transform(self, X):
        X = np.asarray(X, dtype=float).copy()
        mask = np.isnan(X)
        if mask.any():
            for j in range(X.shape[1]):
                X[mask[:, j], j] = self.medians_[j]
            for j in range(X.shape[1]):
                if self.stds_[j] > 0:
                    lo = self.means_[j] - self.sigma_clip * self.stds_[j]
                    hi = self.means_[j] + self.sigma_clip * self.stds_[j]
                    X[:, j] = np.clip(X[:, j], lo, hi)
        return X
```

8.2.2 Eliminación del origen como dato para el entrenamiento

```
def cargar_datos(ruta_csv):
    df = pd.read_csv(ruta_csv)

    # Eliminar columnas de origen
    cols_origen = [c for c in df.columns if c.startswith('origen_')]
    if cols_origen:
        df = df.drop(columns=cols_origen)
        print(f"[+] Columnas one-hot eliminadas: {cols_origen}")

    if 'etiqueta' in df.columns and 'label' not in df.columns:
        df = df.rename(columns={'etiqueta': 'label'})

    # Garantizar que todas las features existan (NaN si falta alguna)
    for col in NOMBRES_FEATURES:
        if col not in df.columns:
            df[col] = np.nan

    n_reales = (df['label'] == 0).sum()
    n_falsos = (df['label'] == 1).sum()
    print(f"\n[+] Dataset: {len(df)} | Reales: {n_reales} | Falsos: {n_falsos}")
```

Figura 45. Eliminar origen como parámetro de entrenamiento

8.2.3 Clasificadores

```
def construir_modelos(sigma_clip=3.0):
    return {
        'rf': Pipeline([('limp', LimpiadorFeatures(sigma_clip)),
                        ('clf', RandomForestClassifier(
                            n_estimators=300, min_samples_leaf=2,
                            class_weight='balanced', random_state=42, n_jobs=-1))]),
        'gbm': Pipeline([('limp', LimpiadorFeatures(sigma_clip)),
                        ('clf', GradientBoostingClassifier(
                            n_estimators=200, learning_rate=0.05,
                            max_depth=4, subsample=0.8, random_state=42))]),
        'lr': Pipeline([('limp', LimpiadorFeatures(sigma_clip)),
                        ('scaler', StandardScaler()),
                        ('clf', LogisticRegression(
                            C=1.0, class_weight='balanced', max_iter=1000))]),
        'xgb': Pipeline([('limp', LimpiadorFeatures(sigma_clip)),
                        ('clf', XGBClassifier(
                            n_estimators=300, learning_rate=0.05, max_depth=5,
                            subsample=0.8, colsample_bytree=0.8,
                            eval_metric='logloss'))]),
    }
```

Figura 46. Clasificadores binarios

8.2.4 Validación cruzada estratificada

```
def evaluar_cv(modelos, X, y):
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    resultados = {}
    print("\n" + "=" * 64)
    print(" CV ESTRATIFICADA 5-FOLD ".center(64, "="))
    print("=" * 64)
    for nombre, modelo in modelos.items():
        s = cross_validate(modelo, X, y, cv=cv,
                          scoring=['accuracy', 'roc_auc', 'f1'], n_jobs=-1)
        resultados[nombre] = {
            'accuracy': s['test_accuracy'].mean(),
            'acc_std': s['test_accuracy'].std(),
            'roc_auc': s['test_roc_auc'].mean(),
            'auc_std': s['test_roc_auc'].std(),
            'f1': s['test_f1'].mean(),
        }
    r = resultados[nombre]
    print(f" [{nombre.upper():4s}] Acc: {r['accuracy']:.3f}±{r['acc_std']:.3f} | "
          f"AUC: {r['roc_auc']:.3f}±{r['auc_std']:.3f} | F1: {r['f1']:.3f}")
    mejor = max(resultados, key=lambda k: resultados[k]['roc_auc'])
    print(f"\n[✓] Mejor según AUC-CV: {mejor.upper()} ({resultados[mejor]['roc_auc']:.3f})")
    return mejor, resultados
```

Figura 47. Evaluación validación cruzada estratificada

8.2.5 LOPO

```
def evaluar_lopo(modelo, X, y, df):
    if 'origen' not in df.columns:
        print("[!] Sin columna 'origen', LOPO omitido")
        return None

    grupos, nombres = pd.factorize(df['origen'])
    logo = LeaveOneGroupOut()

    print("\n" + "=" * 64)
    print(" LEAVE-ONE-PLATFORM-OUT ".center(64, "="))
    print("=" * 64)
    print(" (entrena con N-1 plataformas, testa en la 1 restante)\n")

    resultados = []
    for tr_idx, te_idx in logo.split(X, y, grupos):
        plat = nombres[grupos[te_idx[0]]]
        if len(np.unique(y[te_idx])) < 2:
            print(f" test={plat:18s} | omitido (solo una clase)")
            continue
        if len(te_idx) < 5:
            print(f" test={plat:18s} | omitido (n<5)")
            continue
        m = deepcopy(modelo)
        m.fit(X[tr_idx], y[tr_idx])
        prob = m.predict_proba(X[te_idx])[:, 1]
        pred = m.predict(X[te_idx])
        auc = roc_auc_score(y[te_idx], prob)
        acc = (pred == y[te_idx]).mean()
        f1 = f1_score(y[te_idx], pred, zero_division=0)
        resultados.append({'plat': plat, 'n': len(te_idx),
                           'acc': acc, 'auc': auc, 'f1': f1})
        print(f" test={plat:18s} | n={len(te_idx):4d} | "
              f"acc={acc:.3f} | AUC={auc:.3f} | F1={f1:.3f}")

    if resultados:
        auks = [r['auc'] for r in resultados]
        accs = [r['acc'] for r in resultados]
        print(f"\n → LOPO medio: acc={np.mean(accs):.3f}±{np.std(accs):.3f} | "
              f"AUC={np.mean(auks):.3f}±{np.std(auks):.3f}")
    return resultados
```

Figura 48. Evaluación LOPO

8.3 Modelos por plataforma – entrenar_especializados.py

8.3.1 Filtrar por origen

```
modelos_entrenados = []
for origen in origenes:
    sub = df[df['origen'] == origen]
    n_real = (sub['etiqueta'] == 0).sum() if 'etiqueta' in sub.columns else (sub['label'] == 0).sum()
    n_fake = (sub['etiqueta'] == 1).sum() if 'etiqueta' in sub.columns else (sub['label'] == 1).sum()

    if len(sub) < args.min_muestras:
        print(f"\n[!] '{origen}' tiene {len(sub)} vídeos (<{args.min_muestras}), se omite")
        continue
    if min(n_real, n_fake) < 10:
        print(f"\n[!] '{origen}' tiene clase minoritaria <10 ({n_real}r/{n_fake}f), se omite")
        continue

    # Guardar CSV filtrado
    csv_filtrado = Path(args.salida) / f'features_{origen}.csv'
    sub.to_csv(csv_filtrado, index=False)

    # Modelo de salida
    modelo_salida = Path(args.salida) / f'modelo_{origen}.pkl'
    log_salida = Path(args.salida) / f'log_{origen}.txt'
```

Figura 49. Función para filtrar vídeos por origen

8.3.2 Índice de modelos

```
indice = {
    'modelo_unificado': args.modelo_unificado,
    'modelos_especializados': {
        row['origen']: f"especializados/modelo_{row['origen']}.pkl"
        for _, row in df_res.iterrows()
    }
}
with open(Path(args.salida) / 'indice_modelos.json', 'w', encoding='utf-8') as f:
    json.dump(indice, f, indent=2, ensure_ascii=False)
print(f"[✓] Índice modelos: {Path(args.salida) / 'indice_modelos.json'}")
```

Figura 50. Creación índice de modelos

8.4 Enrutar modelos

```
def seleccionar_modelo(origen, ruta_indice='especializados/indice_modelos.json',
                      fallback='modelo_v2.pkl'):
    indice = cargar_indice(ruta_indice)

    if indice is None:
        return fallback, 'unificado'

    especializados = indice.get('modelos_especializados', {})

    if origen in especializados:
        ruta = especializados[origen]
        if os.path.exists(ruta):
            return ruta, 'especializado'

    # Fallback
    unificado = indice.get('modelo_unificado', fallback)
    return unificado, 'unificado'
```

Figura 51. Función para llamar modelos en base al origen

8.5 Flujo principal y explicabilidad – main.py

Main.py es el código principal de procesamiento de vídeo para emitir veredictos, y primeramente se ha utilizado a través de la consola cuando no había una aplicación web desarrollada para comprobar que todo funcionaba correctamente y hacer las primeras pruebas de ejecución. El código de app.py lo referencia para ejecutar los diferentes análisis.

8.5.1 Detección de origen en producción

```
def detectar_origen(self):
    json_p = os.path.splitext(self.ruta)[0] + '.info.json'
    origen = 'movil_directo'
    if os.path.exists(json_p):
        try:
            ext = json.load(open(json_p, encoding='utf-8')).get('extractor', '').lower()
            origen = next((k for k in ('instagram', 'twitter')
                           if k in ext), 'twitter' if 'x.com' in ext else 'desconocido')
        except Exception:
            origen = 'desconocido'
        print(f"\n[-] Origen detectado (yt-dlp): {origen.upper()}")
    else:
        print(f"\n[-] Sin .info.json → origen asumido: {origen.upper()}")

    self.informe['origen_detectado'] = origen
    for o in ORIGENES:
        self.informe[f'origen_{o}'] = int(origen == o)
    return origen
```

Figura 52. Función para detección de origen en análisis

8.5.2 Carga del modelo y predicción

```
def cargar_modelo(self, ruta_modelo='modelo.pkl'):
    with open(ruta_modelo, 'rb') as f:
        p = pickle.load(f)
    self.ml_modelo = p.get('modelo') or p.get('model')
    self.ml_features = p['feature_names']
    print(f"[-] Modelo ML cargado: {p.get('best_model_name', '?').upper()}")

def predecir_con_modelo(self):
    vec = [float(self.informe.get(f, 0.0)) for f in self.ml_features]
    falt = [f for f in self.ml_features if f not in self.informe]
    if falt:
        print(f"    [-] Features no calculadas (→ 0): {falt}")
    X = np.array([vec])
    prob = float(self.ml_modelo.predict_proba(X)[0][1])
    ver = "IA / SINTÉTICO" if prob > 0.5 else "AUTÉNTICO"
    self.informe['ml_prob_ia'], self.informe['ml_verdicto'] = prob, ver
    print(f"\n[ML] Probabilidad IA: {prob:.1%} → {ver}")
```

Figura 53. Funciones de carga y predicción con modelo

8.5.3 Explicación SHAP

```
clf = self.ml_modelo.named_steps['clf']

# TreeExplainer funciona con GBM, RF y XGBoost
# Para GBM de scikit-learn hay que pasar check_additivity=False
explicador = shap.TreeExplainer(clf)
shap_vals = explicador.shap_values(X, check_additivity=False)

# shap_values puede devolver lista (clasificación binaria) o array
if isinstance(shap_vals, list):
    shap_vals = shap_vals[1]
shap_vals = np.array(shap_vals).reshape(1, -1)

contribuciones = sorted(
    zip(self.ml_features, shap_vals[0]),
    key=lambda x: abs(x[1]), reverse=True
)

print(f"\n{'='*70}")
print(f" POR QUÉ EL MODELO HA DADO {prob:.1%} ".center(70, "="))
print(f"{'='*70}")
print(" (+) empuja hacia IA | (-) empuja hacia AUTÉNTICO\n")

for feat, val in contribuciones[:10]:
    if abs(val) < 0.001:
        continue
    dir_txt = "→ IA " if val > 0 else "→ REAL "
    val_real = self.informe.get(feat, 0.0)
    desc = DESCRIPCIONES.get(feat, "")

    print(f" {'(+)'} if val > 0 else '{(-)}' {feat}")
    print(f" Valor medido : {val_real:.4f} "
          f"Influencia: {val:+.4f} {dir_txt}")
    if desc:
        # Imprimir descripción en líneas de máx 65 caracteres
        palabras, linea = desc.split(), ""
        for p in palabras:
            if len(linea) + len(p) + 1 > 65:
                print(f" {linea}")
                linea = p
            else:
                linea = (linea + " " + p).strip()
        if linea:
            print(f" {linea}")
    print()

self.informe['shap_explicacion'] = [
    {'feature': f, 'valor': float(self.informe.get(f, 0.0)),
     'shap': float(v)}
    for f, v in contribuciones[:10]
]
```

Figura 54. Función para la explicación de *features* con SHAP

8.6 Aplicación web

8.6.1 Activación de eventos

```
# Si el usuario eligió manualmente un origen distinto de 'auto', lo sobrescribe
if origen_manual and origen_manual != 'auto':
    a.informe['origen_detectado'] = origen_manual
    print(f"  [+] Origen manual: {origen_manual}")

# Enrutar al modelo apropiado según origen final
origen_final = a.informe.get('origen_detectado', 'desconocido')
ruta_modelo_usar, tipo_modelo = seleccionar_modelo(origen_final, fallback=RUTA_MODELO)
step(f'Cargando modelo {tipo_modelo} ({origen_final})_', 9)
a.cargar_modelo(ruta_modelo_usar)

step('Analizando metadatos_', 13)
a.analizar_metadatos()
a.analizar_metadatos_forenses()

# Construir argumentos de tiempo para ffmpeg
args_tiempo = []
if segundo_inicio and segundo_inicio > 0:
    args_tiempo += ['-ss', str(segundo_inicio)]
if segundos:
    args_tiempo += ['-t', str(segundos)]

if segundo_inicio and segundo_inicio > 0 and segundos:
    seg_txt = f's{segundo_inicio}-s{segundo_inicio + segundos}'
elif segundo_inicio and segundo_inicio > 0:
    seg_txt = f'desde s{segundo_inicio}'
elif segundos:
    seg_txt = f'primeros {segundos}s'
else:
    seg_txt = 'completo'

step(f'Extrayendo frames ({seg_txt})_', 20)
subprocess.run(
    ['ffmpeg', '-y'] + args_tiempo + ['-i', ruta_video,
    '-vf', 'fps=5,scale=640:-2',
    os.path.join(dir_frames, 'out%d.png')],
    capture_output=True
)
```

Figura 55. Activación de eventos de análisis 1

```
a.frames, i = [], 1
while True:
    p = os.path.join(dir_frames, f'out{i}.png')
    if not os.path.exists(p): break
    img = cv2.imread(p, cv2.IMREAD_GRAYSCALE)
    if img is None: break
    a.frames.append(img)
    i += 1
a.total_frames = len(a.frames)
a.frame_color = cv2.imread(os.path.join(dir_frames, 'out1.png')) if a.total_frames else None
step(f'{a.total_frames} frames cargados', 30)

# Guardar miniatura antes de que se borren los frames
thumb_path = None
if task_id and a.frame_color is not None:
    thumb_path = os.path.join(DIR_TRABAJO, 'thumbs', f'{task_id}.jpg')
    cv2.imwrite(thumb_path, a.frame_color, [cv2.IMWRITE_JPEG_QUALITY, 85])

step('Estructura I/P/B...', 36)
a.analizar_frames_ipb()
step('Nitidez...', 43)
a.analizar_nitidez()
step('Análisis FFT...', 50)
a.analizar_fft()
step('Flujo óptico...', 57)
a.analizar_flujo_optico()
step('Fotometría...', 63)
a.analizar_fotometria()
step('Aberración cromática...', 68)
a.analizar_aberracion_cromatica()
step('Compresión DCT...', 73)
a.analizar_compresion()
step('Ruido...', 78)
a.analizar_ruido()
step('Correlación RGB...', 83)
a.analizar_correlacion_rgb()
step('Modelo ML + SHAP...', 90)
a.predecir_con_modelo()
```

Figura 56. Activación de eventos de análisis 2

8.6.2 Análisis de contexto con Gemini y prompt utilizado

```
def analizar_contexto_gemini(dir_frames, total_frames):
    """
    Manda 4 frames representativos a Gemini 2.5 Flash y pide
    un análisis del contexto visual del vídeo.
    Devuelve None si Gemini no está disponible o falla.
    """
    if not HAY_GEMINI or not GEMINI_API_KEY:
        return None

    try:
        # Seleccionar 4 frames repartidos a lo largo del vídeo
        indices = sorted(set([
            1,
            max(1, total_frames // 3),
            max(1, total_frames * 2 // 3),
            total_frames,
        ]))

        partes = []
        for idx in indices:
            p = os.path.join(dir_frames, f'out{idx}.png')
            if not os.path.exists(p):
                continue
            with open(p, 'rb') as f:
                datos = base64.b64encode(f.read()).decode('utf-8')
            partes.append({'inline_data': {'mime_type': 'image/png', 'data': datos}})

        if not partes:
            return None

        partes.append({'text': """Analiza estas imágenes extraídas de un vídeo y responde SOLO en español.
Tu objetivo es evaluar si el contenido visual podría ser real o generado por inteligencia artificial.

Responde con este formato exacto, una línea por campo:
CONTEXTO: [describe brevemente qué muestra el vídeo en 1-2 frases]
COHERENCIA: [¿es físicamente posible lo que se ve? ¿hay inconsistencias en iluminación, proporciones o física?]
ARTEFACTOS: [¿hay bordes extraños, texturas artificiales, manos/caras deformadas, texto ilegible u otros signos típicos de IA?]
VEREDICTO.CONTEXTO: [escribe solo una de estas opciones: PROBABLE_IA / AMBIGUO / PROBABLE_REAL]
CONFIANZA: [escribe solo una de estas opciones: ALTA / MEDIA / BAJA]
RAZON: [1 frase corta explicando el veredicto]"""})

        modelo = genai.GenerativeModel('gemini-2.5-flash')
        respuesta = modelo.generate_content(partes)
```

Figura 57. Prompt y análisis de contexto con Gemini

8.6.3 Descarga desde URL con yt-dlp

```
def descargar_url(url, dest_dir, q, segundo_inicio=0, segundos=None):
    inicio = segundo_inicio or 0
    duracion = segundos or 30
    fin = inicio + duracion
    if inicio > 0:
        label = f'Descargando vídeo (s{inicio}→s{fin})...'
    else:
        label = f'Descargando vídeo (primeros {duracion}s)...'
    q.put({'type': 'progress', 'label': label, 'pct': 3})
    import yt_dlp
    opts = {
        'outtmpl': os.path.join(dest_dir, '%(id)s.%(ext)s'),
        'format': 'best[ext=mp4]/best',
        'writeinfojson': True,
        'quiet': True,
        'no_warnings': True,
        'download_ranges': yt_dlp.utils.download_range_func(None, [(inicio, fin)]),
        'force_keyframes_at_cuts': True,
        'merge_output_format': 'mp4',
    }
    with yt_dlp.YoutubeDL(opts) as ydl:
        info = ydl.extract_info(url, download=True)
        return ydl.prepare_filename(info)
```

Figura 58. Función de descarga de vídeo a analizar a partir de URL

8.7 Endpoints importantes – app.py

8.7.1 Análisis en segundo plano

```
@app.route('/analyze', methods=['POST'])
def analyze():
    task_id = uuid.uuid4().hex
    q = Queue()
    tareas[task_id] = q
    upload_dir = os.path.join(DIR_TRABAJO, 'uploads')
    dir_frames = os.path.join(DIR_TRABAJO, f'frames_{task_id}')

    # Leer segundos elegidos por el usuario (0 = completo)
    try:
        segundos = int(request.form.get('segundos', 0)) or None
    except Exception:
        segundos = None

    # Segundo de inicio para fragmento personalizado (0 = desde el principio)
    try:
        segundo_inicio = int(request.form.get('segundo_inicio', 0)) or 0
    except Exception:
        segundo_inicio = 0

    # Origen seleccionado manualmente por el usuario ('auto' = detección automática)
    origen_manual = request.form.get('origen_manual', 'auto')

    # Leer archivo/URL dentro del contexto de Flask
    ruta_guardada = None
    url_pegada = None

    if 'file' in request.files and request.files['file'].filename:
        f = request.files['file']
        ruta_guardada = os.path.join(upload_dir, task_id + Path(f.filename).suffix)
        f.save(ruta_guardada)
        videos[task_id] = ruta_guardada # registrar para /video/<task_id>
    elif request.form.get('url'):
        url_pegada = request.form['url'].strip()

    def run():
        try:
            if ruta_guardada:
                ejecutar_analisis(ruta_guardada, dir_frames, q,
                                segundos=segundos, task_id=task_id,
                                origen_manual=origen_manual,
                                segundo_inicio=segundo_inicio)
            elif url_pegada:
                src_dir = os.path.join(upload_dir, task_id)
                os.makedirs(src_dir, exist_ok=True)
                try:
                    ruta = descargar_url(url_pegada, src_dir, q,
                                        segundo_inicio=segundo_inicio,
                                        segundos=segundos)
                    videos[task_id] = ruta
                    ejecutar_analisis(ruta, dir_frames, q,
                                    segundos=segundos, task_id=task_id,
                                    origen_manual=origen_manual,
                                    segundo_inicio=0) # ya recortado por yt-dlp
                except Exception as e:
                    q.put({'type': 'error', 'msg': f'Error descargando: {e}'})
            else:
                q.put({'type': 'error', 'msg': 'Sin archivo ni URL.'})
            finally:
                q.put({'type': 'done'})

        threading.Thread(target=run, daemon=True).start()
    return jsonify({'task_id': task_id})
```

Figura 59. Hilo de ejecución de análisis en segundo plano

Es el punto central: recibe el vídeo, genera un ID único y lanza el análisis en un hilo en segundo plano. El ID lo utiliza el cliente para suscribirse al canal SSE y poder ver el progreso.

También recoge tres parámetros configurables por el usuario: la duración del fragmento a analizar (fija o personalizada), el segundo de inicio del fragmento personalizado y el origen.

8.7.2 Canal de Server Sent Events

```
@app.route('/stream/<task_id>')
def stream(task_id):
    def generate():
        if task_id not in tareas:
            yield f"data: {json.dumps({'type': 'error', 'msg': 'Tarea no encontrada'})}\n\n"
            return
        q = tareas[task_id]
        while True:
            try:
                msg = q.get(timeout=180)
                yield f"data: {json.dumps(msg, default=str)}\n\n"
                if msg.get('type') in ('result', 'error', 'done'):
                    tareas.pop(task_id, None)
                    break
            except Empty:
                yield 'data: {"type": "ping"}\n\n'

    return Response(generate(), mimetype='text/event-stream',
                    headers={'Cache-Control': 'no-cache', 'X-Accel-Buffering': 'no'})
```

Figura 60. Transmisión de eventos de análisis

Emite en tiempo real los eventos del análisis y muestra la barra de progreso del hilo de análisis. El cliente verá cada cambio de etapa (Extraer *frames*, Nitidez...).

8.8 Módulo de análisis facial (Fase Beta)

8.8.1 Eye Aspect Ratio y conteo de parpadeos – *análisis_facial.py*

```
def _ear(lms, idx):
    p = [lms[i] for i in idx]
    a = np.linalg.norm(np.array(p[1]) - np.array(p[5]))
    b = np.linalg.norm(np.array(p[2]) - np.array(p[4]))
    c = np.linalg.norm(np.array(p[0]) - np.array(p[3]))
    return (a + b) / (2.0 * c) if c > 1e-6 else 0.0

def _contar_parpadeos(serie, umbral=0.21):
    count, dentro = 0, False
    for v in serie:
        if v < umbral and not dentro:
            count += 1; dentro = True
        elif v >= umbral:
            dentro = False
    return count
```

Figura 61. Funciones base de análisis facial

8.8.2 Calcular features faciales

```
# -- EAR
ear_i_s.append(_ear(lms, OJO_IZQ_EAR))
ear_d_s.append(_ear(lms, OJO_DER_EAR))

# -- Apertura boca
vert = np.linalg.norm(np.array(lms[BOCA_SUP]) - np.array(lms[BOCA_INF]))
anch = np.linalg.norm(np.array(lms[BOCA_IZQ]) - np.array(lms[BOCA_DER]))
if anch > 1e-6:
    boca_s.append(vert / anch)

# -- Simetria
cx = lms[1][0]
asim = []
for i1, i2 in PARES_SIM:
    d1 = abs(lms[i1][0] - cx)
    d2 = abs(lms[i2][0] - cx)
    if max(d1, d2) > 1e-6:
        asim.append(abs(d1 - d2) / max(d1, d2))
if asim:
    sim_s.append(float(np.mean(asim)))

# -- Temblor de landmarks
lms_arr = np.array(lms)
if lms_ant is not None:
    ancho_cara = np.linalg.norm(np.array(lms[234]) - np.array(lms[454]))
    if ancho_cara > 1e-6:
        mov = np.mean(np.linalg.norm(lms_arr - lms_ant, axis=1))
        temblores.append(mov / ancho_cara)
lms_ant = lms_arr

# -- Contorno
contornos.append(np.array([lms[i] for i in CONTORNO]))

# -- Iluminación cara vs fondo
xs = [int(p[0]) for p in lms]
ys = [int(p[1]) for p in lms]
x0, x1 = max(0, min(xs)), min(w, max(xs))
y0, y1 = max(0, min(ys)), min(h, max(ys))
if x1 > x0 + 10 and y1 > y0 + 10:
    cara_reg = frame[y0:y1, x0:x1]
    mask = np.ones((h, w), dtype=bool)
    mask[y0:y1, x0:x1] = False
    if mask.sum() > 100:
        bc = cv2.cvtColor(cara_reg, cv2.COLOR_BGR2GRAY).mean()
        bf = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)[mask].mean()
        if bf > 1e-6:
            ilum_s.append(abs(bc - bf) / bf)

finally:
    detector.close()
```

Figura 62. Extracción de *features* faciales

8.8.3 Cálculo de features

```
n = len(frames_color)
if n_cara / max(n, 1) < 0.5:
    return {**FEATURES_VACIAS, 'face_detectada': 0.0}

dur = n / max(fps_video, 1)
pb = _contar_parpadeos(ear_i_s)
pd = _contar_parpadeos(ear_d_s)
ppm = (pb + pd) / 2 * (60.0 / max(dur, 1e-6))

todos_ear = ear_i_s + ear_d_s
if todos_ear and ear_i_s:
    ei = np.array(ear_i_s); ed = np.array(ear_d_s)
    denom = (ei + ed).mean() + 1e-6
    sim_parp = float(np.mean(np.abs(ei - ed)) / denom)
else:
    sim_parp = 0.0

if len(contornos) > 1:
    movs = [np.mean(np.linalg.norm(contornos[k] - contornos[k-1], axis=1))
            for k in range(1, len(contornos))]
    borde_est = float(np.mean(movs))
else:
    borde_est = 0.0

return {
    'face_detectada': 1.0,
    'face_parpadeo_freq': float(ppm),
    'face_parpadeo_simetria': float(sim_parp),
    'face_boca_apertura_std': float(np.std(boca_s)) if boca_s else 0.0,
    'face_landmark_temblor': float(np.mean(temblores)) if temblores else 0.0,
    'face_eye_aspect_std': float(np.std(todos_ear)) if todos_ear else 0.0,
    'face_simetria_facial': float(np.mean(sim_s)) if sim_s else 0.0,
    'face_borde_estabilidad': borde_est,
    'face_iluminacion_consist': float(np.std(ilum_s)) if ilum_s else 0.0,
}
```

Figura 63. Cálculo de features